Quadrature Rules and Its Applications

Course Title : Project and Presentation
Course Code : AMTH-460

Submitted By

Class Roll Number : AE-123-008
Registration Number : 2020-616-139
Academic Session : 2023-2024



Submitted To

Department of Applied Mathematics
Faculty of Science
University of Dhaka
Dhaka-1000, Bangladesh

Abstract

The project is about analyzing six Gaussian quadrature rules: Gauss Legendre, Gauss Lobatto, Gauss Radau, Gauss Laguerre, Gauss Chebyshev, and Gauss Hermite, each based on its orthogonal polynomials and weight functions which makes them exact for polynomials up to a set degree. These are checked by numerical runs to see their accuracy and convergence . Gauss Legendre shown with quick convergence and fast precision using few nodes, while Gauss Lobatto stayed reliable and accurate as nodes increased. Gauss Radau did fine at low orders but drifted at higher ones, and Gauss Hermite plus Gauss Chebyshev gave decent results in their weighted ranges, but not standout. Gauss Laguerre had the toughest time, with instability on semi-infinite intervals. In the end, Gauss Legendre and Gauss Lobatto came out on top: fast, precise, and dependable. The study shows why these are key tools in math and engineering, and it offers straightforward advice on picking the right one.

Contents

1 Introduction **2**

2 Quadrature Rules **3**

2.1 Gauss–Legendre Quadrature 3

2.2 Gauss–Radau Quadrature 8

2.3 Gauss–Lobatto Quadrature 11

2.4 Gauss–Hermite Quadrature 13

2.5 Gauss–Laguerre Quadrature 17

2.6 Gauss–Chebyshev Quadrature 20

3 Applications of Gaussian Quadrature **23**

3.1 General Lagrange Interpolation (arbitrary n) 23

3.2 2D Bilinear Shape Functions ($n = 4$, denote by N_i) 25

3.3 Evaluation of 1D integral 27

3.4 Evaluation of line integral 29

3.5 Evaluation of the Double Integral by Gaussian Quadrature 33

3.5.1 Rectangular Region 33

3.5.2 Quadrilateral Region 37

3.5.3 Triangular Region 40

3.5.4 Pentagonal Region 43

Conclusion 48

Chapter 1

Introduction

Numerical quadrature is a fundamental technique in applied mathematics, computational science, and engineering, providing a way to approximate integrals that are difficult or impossible to evaluate analytically [Burden2011, Jain2007, Hildebrand1987]. Quadrature methods replace an integral with a weighted sum of function values at optimally chosen points, called nodes, which allows accurate approximation with minimal computational effort [Stroud1966]. Gaussian quadrature is particularly efficient because it uses orthogonal polynomials to determine the nodes and corresponding weights, achieving exact results for polynomials up to degree $2n-1$ [Stroud1966]. The Gauss Legendre rule is widely used for standard finite intervals, while Gauss Lobatto and Gauss Radau are designed to improve accuracy near boundaries [Lewis1991]. For semi infinite or infinite domains, Gauss Laguerre and Gauss Hermite quadratures are applied, for instance, in quantum mechanics, probability theory, and other engineering and physics problems [Burden2011].In addition to one dimensional integrals, Gaussian quadrature has been successfully extended to multidimensional problems, enabling the approximation of double and triple integrals over regular and irregular domains [Hussain2014, Islam2009, Islam2008].Moreover, Gaussian quadrature plays an important role in probabilistic and statistical computations, including numerical evaluation of expected values and moments [Burden2011, Stroud1966]. It is also widely employed in engineering optimization and control problems where integral expressions arise frequently [Jain2007]. Modern implementations often incorporate fast algorithms, such as the Golub Welsch procedure, for the rapid computation of nodes and weights, making the method practical for large scale computational problems. Overall, the method's adaptability, precision, and computational stability ensure it remains a cornerstone technique for both theoretical and practical numerical analysis. Gaussian quadrature is a very useful numerical integration method that approximates a definite integral as a weighted sum of function values at selected nodes:

$$\int_a^b f(x)\,dx \approx \sum_{i=1}^{n} w_i f(x_i),$$

The remarkable efficiency of Gaussian quadrature stems from its connection with orthogonal polynomials. Each Gaussian quadrature rule is associated with a specific set of orthogonal polynomials with respect to a weight function defined over the integration interval. For example, Gauss–Legendre quadrature uses Legendre polynomials with weight function $w(x) = 1$ over $[-1, 1]$, while Gauss–Laguerre quadrature uses Laguerre polynomials with weight $w(x) = e^{-x}$ over $[0, \infty)$

Chapter 2

Quadrature Rules

This work deals with Numerical Quadrature used when we can't find exact integrals. Instead of the original function, we use a polynomial across $[a, b]$, then add up areas of basic figures. At its heart lies the main integration formula,

$$\int_a^b g(x)\,dx \approx \sum_{i=1}^{n} u_i g(x_i).$$

We look into effective methods such as Gaussian Quadrature, fine-tuning weights u_i along with nodes x_i to get precise results using fewer calculations.

2.1 Gauss–Legendre Quadrature

The Gauss–Legendre Quadrature is a numerical integration method that approximates a definite integral using optimally chosen nodes (the roots of Legendre polynomials) and corresponding weights.

$$\int_{-1}^{1} g(x)\,dx \approx \sum_{i=1}^{n} u_i\, g(x_i)$$

This method is exact for all polynomials up to degree $2n - 1$, providing high accuracy with fewer evaluation points.

Exactness of Gaussian Quadrature

For $N = 2n - 1$, there exist Gaussian points x_i and weights u_i such that

$$\int_{-1}^{1} x^k\,dx = \sum_{i=1}^{n} u_i\,(x_i)^k$$

where $k = 0, 1, 2, \ldots, N$.

Proof

Let the Gaussian points x_i be the roots of the Legendre polynomial of degree n, denoted $P_n(x)$. Let $P(x)$ be an arbitrary polynomial with degree $\leq 2n - 1$.

By the Division Algorithm for Polynomials, we can write $P(x)$ as:

$$P(x) = P_n(x)Q(x) + R(x)$$

where $Q(x)$ and $R(x)$ are polynomials with degrees $\leq n - 1$.

The definition of $P_n(x)$ based on its roots is:

$$P_n(x) = \prod_{i=1}^{n}(x - x_i)$$

Since the Gaussian points x_i are the roots, we have $P_n(x_i) = 0$. Evaluating $P(x)$ at these points yields:

$$P(x_i) = P_n(x_i)Q(x_i) + R(x_i) \implies P(x_i) = R(x_i)$$

This establishes the key relationship between the polynomial and the remainder.

Now, we integrate the polynomial $P(x)$ over the interval $[-1, 1]$:

$$\begin{aligned}\int_{-1}^{1} P(x)\,dx &= \int_{-1}^{1} [P_n(x)Q(x) + R(x)]\,dx \\ &= \int_{-1}^{1} P_n(x)Q(x)\,dx + \int_{-1}^{1} R(x)\,dx\end{aligned}$$

Since $P_n(x)$ is a Legendre polynomial (orthogonal with respect to the weight function $w(x) = 1$), and $\deg(Q) \leq n-1$, the **Orthogonality property** ensures that the first integral is zero:

$$\int_{-1}^{1} P_n(x)Q(x)\,dx = 0$$

Therefore, the integral simplifies to:

$$\int_{-1}^{1} P(x)\,dx = \int_{-1}^{1} R(x)\,dx$$

The polynomial $R(x)$, with $\deg(R) \leq n-1$, can be exactly integrated using the n-point quadrature rule because the weights u_i are chosen to exactly integrate all polynomials of degree $\leq n-1$. Specifically, $R(x)$ can be expressed using the Lagrange basis polynomials $L_i(x)$:

$$\begin{aligned}\int_{-1}^{1} R(x)\,dx &= \int_{-1}^{1} \sum_{i=1}^{n} R(x_i)L_i(x)\,dx \\ &= \sum_{i=1}^{n} R(x_i) \int_{-1}^{1} L_i(x)\,dx \\ &= \sum_{i=1}^{n} R(x_i)u_i\end{aligned}$$

Finally, substituting $R(x_i) = P(x_i)$ from above, we get:

$$\int_{-1}^{1} P(x)\,dx = \sum_{i=1}^{n} P(x_i)u_i$$

This proves that the n-point Gaussian Quadrature rule is exact for an arbitrary polynomial $P(x)$ with degree $\leq 2n-1$.

2-Point Rule

For the 2-point rule over the canonical interval $[-1, 1]$, the approximation is:

$$\int_{-1}^{1} g(x)\,dx \approx u_1 g(x_1) + u_2 g(x_2) \tag{2.1}$$

To find the four unknown parameters (u_1, u_2, x_1, x_2), we enforce that the rule is exact for polynomials of degree up to $2n - 1 = 3$. This gives us a system of four equations using the basis functions $1, x, x^2, x^3$. [Jain2007]

System of Governing Equations

$$\left.\begin{aligned} u_1 + u_2 &= 2 \\ u_1 x_1 + u_2 x_2 &= 0 \\ u_1 x_1^2 + u_2 x_2^2 &= \tfrac{2}{3} \\ u_1 x_1^3 + u_2 x_2^3 &= 0 \end{aligned}\right\} \tag{2.2}$$

Due to the symmetry of the interval $[-1, 1]$, we can assume $x_1 = -x_2$ and $u_1 = u_2$. From the first equation in system (2.2), substituting $u_1 = u_2$:

$$\begin{aligned} 2u_1 = 2 &\implies u_1 = 1 \\ \therefore u_2 &= 1 \end{aligned}$$

From the third equation in system (2.2), substituting the weights and $x_1 = -x_2$:

$$\begin{aligned} x_1^2 + (-x_1)^2 &= \frac{2}{3} \\ \implies 2x_1^2 &= \frac{2}{3} \\ \implies x_1^2 &= \frac{1}{3} \\ \implies x_1 &= \pm\frac{1}{\sqrt{3}} \end{aligned}$$

We choose $x_1 = -\frac{1}{\sqrt{3}}$ and $x_2 = \frac{1}{\sqrt{3}}$.
Substituting the nodes and weights into (2.1):

$$\int_{-1}^{1} g(x)\,dx \approx f\left(-\frac{1}{\sqrt{3}}\right) + f\left(\frac{1}{\sqrt{3}}\right)$$

3-Point Rule

The integral approximation for the 3-point rule is given by:

$$\int_{-1}^{1} g(x)\,dx \approx u_1 g(x_1) + u_2 g(x_2) + u_3 g(x_3) \tag{2.3}$$

Here we have 6 unknowns (3 nodes, 3 weights), allowing for exactness up to degree $2n-1=5$.

System of Governing Equations

The formula must be exact for $g(x)=1,x,x^2,x^3,x^4,x^5$:

$$\left.\begin{aligned} u_1+u_2+u_3 &= 2\\ u_1x_1+u_2x_2+u_3x_3 &= 0\\ u_1x_1^2+u_2x_2^2+u_3x_3^2 &= \tfrac{2}{3}\\ u_1x_1^3+u_2x_2^3+u_3x_3^3 &= 0\\ u_1x_1^4+u_2x_2^4+u_3x_3^4 &= \tfrac{2}{5}\\ u_1x_1^5+u_2x_2^5+u_3x_3^5 &= 0 \end{aligned}\right\} \tag{2.4}$$

By symmetry of the interval, we assume: $x_2=0$, $x_3=-x_1$, and $u_3=u_1$. This simplifies the system significantly.

Finding the Nodes (x_i)

The even-powered equations simplify:

$$\left.\begin{aligned} 2u_1+u_2 &= 2\\ 2u_1x_1^2 &= \tfrac{2}{3}\\ 2u_1x_1^4 &= \tfrac{2}{5} \end{aligned}\right\} \tag{2.5}$$

Dividing the third by the second to eliminate u_1:

$$\frac{2u_1x_1^4}{2u_1x_1^2}=\frac{2/5}{2/3}$$
$$x_1^2=\frac{3}{5} \implies x_1=\pm\sqrt{\frac{3}{5}}$$

Therefore, the nodes are: $x_1=-\sqrt{\frac{3}{5}}$, $x_2=0$, and $x_3=\sqrt{\frac{3}{5}}$.

Finding the Weights (u_i)

Substitute $x_1^2=\frac{3}{5}$ into the second equation of system (2.5):

$$2u_1\left(\frac{3}{5}\right)=\frac{2}{3} \implies u_1=\frac{1}{3}\cdot\frac{5}{3}=\frac{5}{9}$$

Since $u_3=u_1$, we have $u_3=\frac{5}{9}$.

Finally, use the first equation of system (2.5) to find u_2:

$$2\left(\frac{5}{9}\right) + u_2 = 2$$
$$\frac{10}{9} + u_2 = 2 \implies u_2 = 2 - \frac{10}{9} = \frac{8}{9}$$

Substituting the nodes and weights into the quadrature formula (2.3):

$$\int_{-1}^{1} g(x)\,dx \approx \frac{5}{9} f\left(-\sqrt{\frac{3}{5}}\right) + \frac{8}{9} g(0) + \frac{5}{9} f\left(\sqrt{\frac{3}{5}}\right)$$

Error Analysis

If f is $2n$-times continuously differentiable on $[-1, 1]$, then there exists $\xi \in (-1, 1)$ such that

$$E_n(f) = \int_{-1}^{1} g(x)\,dx - \sum_{i=1}^{n} u_i g(x_i) = C_n f^{(2n)}(\xi), \tag{2.6}$$

where the error constant is

$$C_n = \frac{2^{2n+1}(n!)^4}{(2n+1)\big[(2n)!\big]^3}. \tag{2.7}$$

Thus the error is proportional to the $(2n)$-th derivative evaluated at some point in the interval.

Numeric values of C_n for small n:

- $n = 1$: $C_1 = \frac{1}{3} \approx 0.333333$
- $n = 2$: $C_2 \approx 0.0074074074$
- $n = 3$: $C_3 \approx 6.3492 \times 10^{-5}$
- $n = 4$: $C_4 \approx 2.8795 \times 10^{-7}$

Code for Gaussian Quadrature

```
import numpy as np
from numpy.linalg import eigh

def gauss_legendre_golub_welsch(n):

    diag_a = np.zeros(n)
    k = np.arange(1, n)
    beta = k**2 / (4 * k**2 - 1)
    diag_b = np.sqrt(beta)
```

```
    J = np.diag(diag_a) + np.diag(diag_b, k=1) + np.diag(diag_b, k=-1)
    eigenvalues, eigenvectors = eigh(J)
    nodes = eigenvalues
    mu_0 = 2.0
    weights = mu_0 * (eigenvectors[0, :])**2
    return nodes, weights

n = 3
x, w = gauss_legendre_golub_welsch(n)

print(f"Number of points (n): {n}")
print(f"Nodes (x_i): {x}")
print(f"Weights (u_i): {w}")
```

Listing 2.1: Golub-Welsch Algorithm for Gauss-Legendre Quadrature

Table 2.1: Nodes (x_i) and Weights (u_i) for n-Point Gauss–Legendre Rules

n	**Nodes (x_i)**		**Weights (u_i)**	
	Exact Form	**Decimal (10 Places)**	**Exact Form**	**Decimal (10 Places)**
1	0	0.0000000000	2	2.0000000000
2	$\pm\frac{1}{\sqrt{3}}$	$\pm$0.5773502692	1	1.0000000000
3	0	0.0000000000	$\frac{8}{9}$	0.8888888889
	$\pm\sqrt{\frac{3}{5}}$	$\pm$0.7745966692	$\frac{5}{9}$	0.5555555556
4	$\pm\sqrt{\frac{3}{7}-\frac{2}{7}\sqrt{\frac{6}{5}}}$	$\pm$0.3399810436	$\frac{18+\sqrt{30}}{36}$	0.6521451549
	$\pm\sqrt{\frac{3}{7}+\frac{2}{7}\sqrt{\frac{6}{5}}}$	$\pm$0.8611363116	$\frac{18-\sqrt{30}}{36}$	0.3478548451

2.2 Gauss–Radau Quadrature

The Gauss-Radau quadrature rule is a variation of Gaussian quadrature where one of the endpoints of the integration interval $[-1, 1]$ is required to be a node. [Jain2007] Assuming the left endpoint, $\mathbf{x_1 = -1}$, is fixed:

$$\int_{-1}^{1} g(x)\,dx = u_1 g(-1) + \sum_{i=2}^{n} u_i g(x_i) \tag{2.8}$$

For n total nodes, there are **2n-1** unknowns (**n-1** variable nodes, **n** weights). This structure allows the rule to be exact for polynomials of degree up to $\mathbf{2n - 2}$.

3-Point Rule

For $n = 3$, the rule is exact for polynomials up to degree 4. The approximation is:

$$\int_{-1}^{1} g(x)\,dx \approx u_1 g(-1) + u_2 g(x_2) + u_3 g(x_3) \tag{2.9}$$

System of Governing Equations

The formula must be exact for $g(x) = 1, x, x^2, x^3, x^4$:

$$\left.\begin{aligned} u_1 + u_2 + u_3 &= 2 \\ -u_1 + u_2 x_2 + u_3 x_3 &= 0 \\ u_1 + u_2 x_2^2 + u_3 x_3^2 &= \tfrac{2}{3} \\ -u_1 + u_2 x_2^3 + u_3 x_3^3 &= 0 \\ u_1 + u_2 x_2^4 + u_3 x_3^4 &= \tfrac{2}{5} \end{aligned}\right\} \tag{2.10}$$

Solution

The nodes x_2, x_3 are the roots of the polynomial $P_3(x) - P_2(x)$, where $P_k(x)$ is the Legendre polynomial of degree k. The solutions are found to be:

$$x_1 = -1, \quad x_2 = \frac{1 - \sqrt{6}}{5}, \quad x_3 = \frac{1 + \sqrt{6}}{5}$$

The corresponding weights are:

$$u_1 = \frac{2}{9}, \quad u_2 = \frac{16 + \sqrt{6}}{18}, \quad u_3 = \frac{16 - \sqrt{6}}{18}$$

Substituting the nodes and weights into (2.9) yields the final formula:

$$\int_{-1}^{1} g(x)\,dx \approx \frac{2}{9} g(-1) + \frac{16 + \sqrt{6}}{18} f\left(\frac{1 - \sqrt{6}}{5}\right) + \frac{16 - \sqrt{6}}{18} f\left(\frac{1 + \sqrt{6}}{5}\right)$$

Error Analysis

The error (remainder term) for the n-point Radau rule (with $x = -1$ fixed) is a function of the $(2n-1)$-th derivative of $g(x)$.

For $f \in C^{2n-1}[-1, 1]$, the error $R_n[f]$ is:

$$R_n[f] = C_n f^{(2n-1)}(\xi), \quad \xi \in (-1, 1)$$

where C_n is a constant related to the integral of the squared polynomial product over the weighted interval.

Code for Radau Quadrature

```python
import numpy as np
from numpy.polynomial.legendre import Legendre

def gauss_radau_quadrature(n):
    # 1. Compute nodes and weights

    Pn_1 = Legendre.basis(n - 1)
    Pn = Legendre.basis(n)
    num_poly = (Pn_1 + Pn).convert(kind=np.polynomial.Polynomial)
    quotient, remainder = np.polynomial.polynomial.polydiv(num_poly.coef, [1, 1])
    Q_poly = np.polynomial.Polynomial(quotient)
    roots = np.roots(Q_poly.coef[::-1])
    roots = np.sort(np.real_if_close(roots))
    x = np.concatenate(([-1.0], roots))
    w = np.zeros_like(x)
    w[0] = 2 / n**2

    for i in range(1, n):
        xi = x[i]
        Pn_1_xi = Pn_1(xi)
        w[i] = (1 - xi) / (n**2 * (Pn_1_xi**2))
    return x, w

N_points = 3
nodes, weights = gauss_radau_quadrature(N_points)
print(f"N={N_points}Gauss-Radau-RuleFixedx=-1)")
print("Nodes:", nodes)
print("Weights:", weights)
```

Listing 2.2: Code for Radau Quadrature

Table 2.2: Nodes (x_i) and Weights (u_i) for n-Point Radau Quadrature

n	**Nodes** (x_i)	**Weights** (u_i)
2	$x_1 = -1$	$u_1 = \frac{1}{4} = 0.25$
	$x_2 = \frac{1}{3} \approx 0.3333$	$u_2 = \frac{3}{4} = 0.75$
3	$x_1 = -1$	$u_1 = \frac{2}{9} \approx 0.2222$
	$x_2 = \frac{-1+\sqrt{6}}{5} \approx 0.2899$	$u_2 = \frac{16+\sqrt{6}}{18} \approx 1.0250$
	$x_3 = \frac{1+\sqrt{6}}{5} \approx 0.6899$	$u_3 = \frac{16-\sqrt{6}}{18} \approx 0.7528$

2.3 Gauss–Lobatto Quadrature

The Gauss-Lobatto quadrature rule is a form of closed Gaussian quadrature where both endpoints, $\mathbf{x_1 = -1}$ and $\mathbf{x_n = 1}$, are included as fixed nodes. [Jain2007] The general approximation form is:

$$\int_{-1}^{1} g(x)\,dx = u_1 g(-1) + u_n g(1) + \sum_{i=2}^{n-1} u_i g(x_i) \tag{2.11}$$

With n total nodes, there are **2n-2** unknown parameters (**n-2** variable nodes, **n** weights). This structure ensures the method is exact for polynomials of degree up to $\mathbf{2n-3}$.

3-Point Rule

For $n = 3$, the rule is exact for polynomials up to degree $2n - 3 = 3$. We have four unknown parameters: u_1, u_2, u_3, x_2. The approximation is:

$$\int_{-1}^{1} g(x)\,dx = u_1 g(-1) + u_3 g(1) + u_2 g(x_2) \tag{2.12}$$

System of Governing Equations

We enforce exactness for the basis functions $g(x) = 1, x, x^2, x^3$:

$$\left.\begin{aligned} u_1 + u_2 + u_3 &= 2 \\ -u_1 + u_2 x_2 + u_3 x_3 &= 0 \\ u_1 + u_2 x_2^2 + u_3 x_3^2 &= \tfrac{2}{3} \\ -u_1 + u_2 x_2^3 + u_3 x_3^3 &= 0 \\ u_1 + u_2 x_2^4 + u_3 x_3^4 &= \tfrac{2}{5} \end{aligned}\right\} \tag{2.13}$$

Solution

Due to the symmetry of the interval and moments:

1. From (2.13) , we deduce that $u_2 x_2 = u_2 x_2^3$. Since $u_2 \neq 0$, we must have $x_2 = x_2^3$, which implies $\mathbf{x_2 = 0}$ or $x_2 = \pm 1$.
2. Substituting $x_2 = 0$ into third equation of (2.13): $u_1 + 0 + u_3 = \frac{2}{3}$.
3. Substituting $u_1 + u_3 = \frac{2}{3}$ into first equation of (2.13) $\frac{2}{3} + u_2 = 2 \implies \mathbf{u_2 = \frac{4}{3}}$.
4. Substituting u_2 and x_2 into (2.13) gives $-u_1 + u_3 = 0$, so $\mathbf{u_1 = u_3}$.
5. Since $u_1 + u_3 = \frac{2}{3}$, we have $2u_1 = \frac{2}{3} \implies \mathbf{u_1 = u_3 = \frac{1}{3}}$.

Substituting the nodes and weights into (2.12) yields:

$$\int_{-1}^{1} g(x)\,dx \approx \frac{1}{3}g(-1) + \frac{4}{3}g(0) + \frac{1}{3}g(1) = \frac{1}{3}\left[g(-1) + 4g(0) + g(1)\right]$$

Error Analysis

The error (or remainder term), $E_n(f)$, for the n-point Lobatto rule is structurally tied to the $2n-2$ derivative of the integrand $g(x)$.

For a smooth function $f \in C^{2n-2}([-1,1])$, the error is:

$$E_n(f) = C_n^{(L)} f^{(2n-2)}(\xi), \quad \xi \in (-1,1)$$

where $C_n^{(L)}$ is a constant determined by the specific Lobatto rule, and ξ is some point within the integration interval.

Code for Lobatto Quadrature

The Lobatto nodes are the roots of the Legendre polynomial of degree $n-1$, $P_{n-1}(x)$, plus the endpoints ± 1. The variable nodes (x_2 to x_{n-1}) are the roots of the derivative of the Legendre polynomial, $P'_{n-1}(x)$, which are related to the Jacobi polynomials $P_{n-2}^{(1,1)}(x)$.

```
1  import numpy as np
2  from numpy.polynomial.legendre import Legendre
3
4  def gauss_lobatto_nodes_weights(n):
5      if n < 2:
6          raise ValueError("Number␣of␣nodes␣must␣be␣at␣least␣2␣for␣↩
       GaussLobatto␣rule.")
7
8      Pn_1 = Legendre.basis(n-1)
9
```

```
    if n == 2:
        nodes = np.array([-1.0, 1.0])
    else:
        dPn_1 = Pn_1.deriv()
        interior = dPn_1.roots()
        interior = np.real_if_close(interior, tol=1000)
        nodes = np.concatenate(([-1.0], np.sort(interior), [1.0]))

    Pvals = Pn_1(nodes)
    Pvals = np.real_if_close(Pvals, tol=1000)
    weights = 2.0 / (n * (n - 1) * (Pvals ** 2))

    return np.asarray(nodes, dtype=float), np.asarray(weights, dtype=float)

n=4
x_nodes, wx = gauss_lobatto_nodes_weights(n)
print(f"Gauss-Lobatto nodes (n={n}): {x_nodes}")
print(f"Gauss-Lobatto weights (n={n}): {wx}")
```

Listing 2.3: Code for Lobatto Quadrature

Table 2.3: Nodes (x_i) and Weights (u_i) for Lobatto Quadrature

n	**Nodes** (x_i)	**Weights** (u_i)
2	$x_1 = -1$	$u_1 = \frac{1}{4} = 0.25$
	$x_2 = \frac{1}{3} \approx 0.3333$	$u_2 = \frac{3}{4} = 0.75$
3	$x_1 = -1$	$u_1 = \frac{2}{9} \approx 0.2222$
	$x_2 = \frac{-1+\sqrt{6}}{5} \approx 0.2899$	$u_2 = \frac{16+\sqrt{6}}{18} \approx 1.0250$
	$x_3 = \frac{1+\sqrt{6}}{5} \approx 0.6899$	$u_3 = \frac{16-\sqrt{6}}{18} \approx 0.7528$

2.4 Gauss–Hermite Quadrature

The Gauss-Hermite quadrature rule is a powerful numerical technique for approximating integrals over the infinite interval $(-\infty, \infty)$ [Jain2007]where the integrand includes a standard Gaussian weighting function, $w(x) = e^{-x^2}$.

$$\int_{-\infty}^{\infty} e^{-x^2} g(x)\, dx \approx \sum_{i=1}^{n} u_i g(x_i) \tag{2.14}$$

The nodes (x_i) are the roots of the Hermite polynomial $H_n(x)$, and the weights (u_i) are determined such that the rule is exact for polynomials $g(x)$ up to degree $2n-1$. The Gauss-Hermite quadrature formula is:

$$\int_{-\infty}^{\infty} g(x)e^{-x^2}dx = \sum_{k=1}^{n} A_k g(x_k)$$

Transformation Method

For the integral transformation:

$$\int_a^b g(x)dx \to \int_{-1}^{1} g(t)dt$$

Using the hyperbolic tangent transformation:

$$x = \frac{b-a}{2}\tanh(t) + \frac{a+b}{2}$$

The derivative is:

$$\frac{dx}{dt} = \frac{b-a}{2}\operatorname{sech}^2(t)$$

Therefore, the transformed function becomes:

$$g(t) = g(x(t))\frac{dx}{dt} = f\left(\frac{b-a}{2}\tanh(t) + \frac{a+b}{2}\right)\cdot\frac{b-a}{2}\operatorname{sech}^2(t)$$

3-Point Rule

The 3-point rule involves 6 unknowns, allowing for exactness up to degree $2n-1=5$. The integral approximation is:

$$\int_{-\infty}^{\infty} e^{-x^2} g(x)\,dx \approx u_1 g(x_1) + u_2 g(x_2) + u_3 g(x_3) \tag{2.15}$$

System of Governing Equations

The formula must be exact for $g(x) = 1, x, x^2, x^3, x^4, x^5$:

$$\left.\begin{aligned} u_1 + u_2 + u_3 &= \sqrt{\pi} \\ u_1x_1 + u_2x_2 + u_3x_3 &= 0 \\ u_1x_1^2 + u_2x_2^2 + u_3x_3^2 &= \frac{\sqrt{\pi}}{2} \\ u_1x_1^4 + u_2x_2^4 + u_3x_3^4 &= \frac{3\sqrt{\pi}}{4} \end{aligned}\right\} \tag{2.16}$$

Equations for $g(x) = x^3$ and $g(x) = x^5$ also equal zero and are redundant under symmetry.

Finding the Nodes and Weights (x_i, u_i) via Symmetry

We assume symmetry: $\mathbf{x_2 = 0}$, $\mathbf{x_3 = -x_1}$, and $\mathbf{u_3 = u_1}$.

The even-powered equations from (2.16) simplify as follows:

$$2u_1x_1^2 = \frac{\sqrt{\pi}}{2} \tag{2.17}$$

$$2u_1x_1^4 = \frac{3\sqrt{\pi}}{4} \tag{2.18}$$

Dividing (2.18) by (2.17) eliminates u_1:

$$\begin{aligned} \frac{2u_1x_1^4}{2u_1x_1^2} &= \frac{3\sqrt{\pi}/4}{\sqrt{\pi}/2} \\ x_1^2 &= \frac{3}{4} \cdot 2 = \frac{3}{2} \\ \implies x_1 &= \pm\sqrt{\frac{3}{2}} \end{aligned}$$

Thus, the nodes are:

$$\mathbf{x_1} = -\sqrt{\frac{\mathbf{3}}{\mathbf{2}}}, \quad \mathbf{x_2 = 0}, \quad \mathbf{x_3} = \sqrt{\frac{\mathbf{3}}{\mathbf{2}}}.$$

Substitute $x_1^2 = 3/2$ into (2.17) to find u_1:

$$\begin{aligned} 2u_1\left(\frac{3}{2}\right) &= \frac{\sqrt{\pi}}{2} \\ 3u_1 = \frac{\sqrt{\pi}}{2} &\implies u_1 = \frac{\sqrt{\pi}}{6}. \end{aligned}$$

By symmetry, $\mathbf{u_3 = u_1} = \sqrt{\pi}/\mathbf{6}$.

Finally, using first equation from (2.16):

$$\begin{aligned} u_1 + u_2 + u_3 &= \sqrt{\pi} \\ \implies 2u_1 + u_2 &= \sqrt{\pi} \\ \implies 2\left(\frac{\sqrt{\pi}}{6}\right) + u_2 &= \sqrt{\pi} \\ \implies u_2 &= \sqrt{\pi} - \frac{\sqrt{\pi}}{3} = \frac{2\sqrt{\pi}}{3}. \end{aligned}$$

The weights are:

$$\mathbf{u_1} = \frac{\sqrt{\pi}}{\mathbf{6}}, \quad \mathbf{u_2} = \frac{\mathbf{2}\sqrt{\pi}}{\mathbf{3}}, \quad \mathbf{u_3} = \frac{\sqrt{\pi}}{\mathbf{6}}.$$

Substituting the nodes and weights into (2.15) yields:

$$\int_{-\infty}^{\infty} e^{-x^2} g(x)\,dx \approx \frac{\sqrt{\pi}}{6} f\left(-\sqrt{\frac{3}{2}}\right) + \frac{2\sqrt{\pi}}{3} g(0) + \frac{\sqrt{\pi}}{6} f\left(\sqrt{\frac{3}{2}}\right)$$

Error Analysis

If $f \in O^{2n}(\mathbb{R})$, the error (remainder term) for the n-point Gauss-Hermite rule is proportional to the $2n$-th derivative evaluated at some point ξ:

$$R_n[f] = \int_{-\infty}^{\infty} e^{-x^2} g(x)\, dx - \sum_{i=1}^{n} u_i g(x_i) = C_n f^{(2n)}(\xi)$$

where the constant C_n is:

$$C_n = \frac{n!\sqrt{\pi}}{2^n (2n)!}$$

The rapid decrease of C_n with increasing n illustrates the high convergence rate of the Gauss-Hermite method.

Code for Gauss-Hermite Quadrature

The most robust way to find higher-order nodes and weights is using the relationship with the eigenvalues of the tridiagonal matrix defined by the recurrence relation for Hermite polynomials (the Golub-Welsch algorithm).

```
import numpy as np
from numpy.polynomial.hermite import hermgauss

def calculate_gauss_hermite(N):
    """
␣␣␣␣Returns␣nodes␣and␣weights␣for␣Gauss--Hermite␣quadrature:
␣␣␣␣@\int_{-\infty}^{\infty}␣e^{-x^2}␣g(x)␣\,␣dx␣\approx␣\sum_{i=1}^n␣u_i␣↩
    g(x_i)@
␣␣␣␣"""
    if N < 1:
        raise ValueError("N␣must␣be␣at␣least␣1.")
    nodes, weights = hermgauss(N)
    return nodes, weights

# Example: 3-point rule
N_points = 3
nodes, weights = calculate_gauss_hermite(N_points)
print(f"Number␣of␣points␣(n):␣{N_points}")
print(f"Nodes␣(x_i):␣{nodes}")
print(f"Weights␣(u_i):␣{weights}")
```

Listing 2.4: Python Code for Gauss–Hermite Quadrature Nodes and Weights

Table 2.4: Nodes (x_i) and Weights (u_i) for n-Point Gauss–Hermite Quadrature

n	**Nodes** (x_i)	**Weights** (u_i)
1	0	$\sqrt{\pi}$
2	$\pm\frac{1}{\sqrt{2}}$	$\frac{\sqrt{\pi}}{2}$
3	0	$\frac{2\sqrt{\pi}}{3}$
	$\pm\sqrt{\frac{3}{2}}$	$\frac{\sqrt{\pi}}{6}$

2.5 Gauss–Laguerre Quadrature

The Gauss-Laguerre quadrature rule is used to approximate integrals over the semi-infinite interval $[0, \infty)$ [Jain2007]involving the exponential weighting function, $w(x) = e^{-x}$:

$$\int_0^\infty e^{-x} g(x)\, dx \approx \sum_{i=1}^{n} u_i g(x_i) \tag{2.19}$$

Here, n is the number of nodes. The rule is exact when the function $g(x)$ is a polynomial of degree up to $2n - 1$.

The standard Gauss-Laguerre quadrature integrates functions of the form:

$$\int_0^\infty g(x) e^{-x} dx$$

Transformation Method

To transform a finite interval integral:

$$\int_a^b g(x) dx \to \int_0^\infty g(t) e^{-t} dt$$

Use the rational transformation:

$$x = a + (b - a)\frac{t}{1 + t}$$

The corresponding derivative is:

$$\frac{dx}{dt} = \frac{b - a}{(1 + t)^2}$$

The transformed function becomes:

$$g(t) = g(x(t))\frac{dx}{dt} = f\left(a + (b - a)\frac{t}{1 + t}\right) \cdot \frac{b - a}{(1 + t)^2}$$

For the Gauss-Laguerre application, multiply by e^t:

$$g(t) = f\left(a + (b-a)\frac{t}{1+t}\right) \cdot \frac{b-a}{(1+t)^2} \cdot e^t$$

3-Point Rule

The 3-point rule requires 6 equations for exactness up to degree $2n-1 = 5$. The moments are:

$$\int_0^\infty x^k e^{-x} dx = k!$$

The system of governing equations for the 6 unknowns $(u_1, u_2, u_3, x_1, x_2, x_3)$ is:

$$\left.\begin{aligned} u_1 + u_2 + u_3 &= 1 \\ u_1x_1 + u_2x_2 + u_3x_3 &= 1 \\ u_1x_1^2 + u_2x_2^2 + u_3x_3^2 &= 2 \\ u_1x_1^3 + u_2x_2^3 + u_3x_3^3 &= 6 \\ u_1x_1^4 + u_2x_2^4 + u_3x_3^4 &= 24 \\ u_1x_1^5 + u_2x_2^5 + u_3x_3^5 &= 120 \end{aligned}\right\} \tag{2.20}$$

The nodes x_i are the roots of the Laguerre polynomial $L_3(x) = -x^3 + 9x^2 - 18x + 6 = 0$. Solving the cubic equation gives the three positive roots:

$$x_1 \approx 0.41577456, \quad x_2 \approx 2.29428036, \quad x_3 \approx 6.28994508.$$

With these nodes fixed, the system (2.20) reduces to three linear equations in the weights u_1, u_2, u_3:

$$\left.\begin{aligned} u_1 + u_2 + u_3 &= 1 \\ u_1x_1 + u_2x_2 + u_3x_3 &= 1 \\ u_1x_1^2 + u_2x_2^2 + u_3x_3^2 &= 2. \end{aligned}\right\} \tag{2.21}$$

Solving this system yields the weights:

$$u_1 \approx 0.71109301, \quad u_2 \approx 0.27851773, \quad u_3 \approx 0.01038926.$$

The final 3-point Gauss–Laguerre quadrature rule is therefore:

$$\int_0^\infty g(x)e^{-x}\,dx \approx 0.7111\,g(0.4158) + 0.2785\,g(2.2943) + 0.0104\,g(6.2899) \tag{2.22}$$

This rule is exact for all polynomials $g(x)$ of degree at most 5.

Error Analysis

The error $R_n[f]$ for the n-point Gauss-Laguerre rule provides a measure of its accuracy.

For a sufficiently smooth function f (i.e., $f \in O^{2n}([0,\infty))$), the error is given by:

$$R_n[f] = \int_0^\infty e^{-x} g(x)\, dx - \sum_{i=1}^{n} u_i g(x_i) = C_n f^{(2n)}(\xi)$$

where $\xi \in (0,\infty)$.

The constant C_n for the Laguerre rule is:

$$C_n = \frac{(n!)^2}{(2n)!}$$

Code for Gaussian Laguerre

The `laggauss` function from NumPy provides a stable way to calculate the nodes and weights using numerical methods based on the properties of Laguerre polynomials.

```
import numpy as np
from numpy.polynomial.laguerre import laggauss

def calculate_gauss_laguerre(N):
    if N < 1:
        raise ValueError("N must be at least 1 for Gauss-Laguerre quadrature.")

    # laggauss(N) returns nodes and weights for:
    # @\int_{0}^{\infty} e^{-x} g(x) \, dx \approx \sum_{i=1}^n u_i g(x_i)@
    nodes, weights = laggauss(N)
    return nodes, weights

# Example: 3-point rule
N_points = 3
nodes, weights = calculate_gauss_laguerre(N_points)
print(f"Number of points (n): {N_points}")
print(f"Nodes (x_i): {nodes}")
print(f"Weights (u_i): {weights}")
```

Listing 2.5: Python Code for Gauss-Laguerre Quadrature

The table below provides the exact and high-precision decimal values for the nodes and weights for the first few Gauss-Laguerre rules.

Table 2.5: Nodes (x_i) and Weights (u_i) for n-Point Gauss–Laguerre Rules

n	**Nodes** (x_i)	**Weights** (u_i)
1	$x_1 = 1$	$u_1 = 1$
2	$x_1 = 2 - \sqrt{2} \approx 0.5858$	$u_1 = \dfrac{2+\sqrt{2}}{4} \approx 0.8536$
	$x_2 = 2 + \sqrt{2} \approx 3.4142$	$u_2 = \dfrac{2-\sqrt{2}}{4} \approx 0.1464$
3	$x_1 \approx 0.4158$	$u_1 \approx 0.7111$
	$x_2 \approx 2.2943$	$u_2 \approx 0.2785$
	$x_3 \approx 6.2899$	$u_3 \approx 0.0104$

2.6 Gauss–Chebyshev Quadrature

The Gauss-Chebyshev quadrature rule is designed for integrals over the canonical interval $[-1, 1]$ with the specific weighting function $w(x) = \frac{1}{\sqrt{1-x^2}}$:

$$\int_{-1}^{1} \frac{g(x)}{\sqrt{1-x^2}}\, dx \approx \sum_{i=1}^{n} u_i g(x_i) \tag{2.23}$$

This method is notable because the weights u_i are all equal for any n, and the nodes x_i are the roots of the Chebyshev polynomial of the first kind, $T_n(x)$. The rule is exact for polynomials $g(x)$ up to degree $2n - 1$.

3-Point Rule

The 3-point rule requires 6 equations for exactness up to degree $2n - 1 = 5$. The integral moments are required:

$$\int_{-1}^{1} \frac{x^k}{\sqrt{1-x^2}} dx$$

The nodes are the roots of the Chebyshev polynomial $T_3(x) = 4x^3 - 3x$, which are 0 and $\pm\frac{\sqrt{3}}{2}$.

For $n = 3$ Gauss-Chebyshev Quadrature, the nodes (x_i) and weights (u_i) are given by the general formulas:

$$x_i = \cos\left(\frac{(2i-1)\pi}{2n}\right), \quad u_i = \frac{\pi}{n}$$

For $n = 3$, the weights are $\mathbf{u_1} = \mathbf{u_2} = \mathbf{u_3} = \frac{\pi}{\mathbf{3}}$. The nodes are:

$$x_1 = \cos\left(\frac{\pi}{6}\right) = \frac{\sqrt{3}}{2}$$
$$x_2 = \cos\left(\frac{3\pi}{6}\right) = \cos\left(\frac{\pi}{2}\right) = 0$$
$$x_3 = \cos\left(\frac{5\pi}{6}\right) = -\frac{\sqrt{3}}{2}$$

The final formula is:

$$\int_{-1}^{1} \frac{g(x)}{\sqrt{1-x^2}}\,dx \approx \frac{\pi}{3} f\left(-\frac{\sqrt{3}}{2}\right) + \frac{\pi}{3} g(0) + \frac{\pi}{3} f\left(\frac{\sqrt{3}}{2}\right)$$

Error Analysis

The error term $R_n[f]$ quantifies the accuracy of the n-point Gauss-Chebyshev rule (first kind).

For a sufficiently smooth function f (i.e., $f \in O^{2n}([-1,1])$), the error is given by:

$$R_n[f] = \frac{\pi}{2^{2n-1}(2n)!} f^{(2n)}(\xi)$$

where ξ is some point in $(-1,1)$.

Code for Gaussian Chebyshev

The `chebgauss` function in NumPy can generate the nodes and weights using numerical stability methods based on the properties of Chebyshev polynomials.

```
import numpy as np
from numpy.polynomial.chebyshev import chebgauss

def calculate_gauss_chebyshev(N):
    if N < 1:
        raise ValueError("N must be at least 1 for Gauss-Chebyshev quadrature.")

    # chebgauss(N) returns nodes and weights for:
    # @\int_{-1}^{1} \frac{g(x)}{\sqrt{1-x^2}} \, dx \approx \sum_{i=1}^n u_i g(x_i)@
    nodes, weights = chebgauss(N)
    return nodes, weights

# Example: 3-point rule
N_points = 3
nodes, weights = calculate_gauss_chebyshev(N_points)
print(f"Number of points (n): {N_points}")

# Nodes for n=3 should be: \pm \sqrt{3}/2 \approx \pm 0.866025, and 0
print(f"Nodes (x_i): {nodes}")

# Weights for n=3 should be: [\pi/3, \pi/3, \pi/3] \approx [1.047197...]
print(f"Weights (u_i): {weights}")
```

Listing 2.6: Python Code for Gauss-Chebyshev Quadrature

The table below provides the exact values for the nodes and weights for the first few Gauss-Chebyshev rules.

Table 2.6: Nodes (x_i) and Weights (u_i) for n-Point Gauss–Chebyshev Rules

n	**Nodes** (x_i)	**Weights** (u_i)
1	0	π
2	$\pm\frac{1}{\sqrt{2}}$	$\frac{\pi}{2}$
3	0	$\frac{\pi}{3}$
	$\pm\frac{\sqrt{3}}{2}$	$\frac{\pi}{3}$

Chapter 3

Applications of Gaussian Quadrature

In this chapter, we derive **shape functions from Lagrange interpolation polynomials** and use them to **map any physical line or area to a standard reference element** (like the interval $[-1, 1]$ in 1D or the square $[-1, 1] \times [-1, 1]$ in 2D). This mapping lets us handle interpolation, differentiation, and integration the same way no matter the element shape. In finite element methods, these mappings make calculations easier and keep things consistent between the real domain and the reference one.

We consider three types of shape functions:

- 1D **Linear** shape functions (L_i)
- 1D **Quadratic** shape functions (Q_i)
- 2D **Bilinear** shape functions (N_i)

Each is derived using the same Lagrange interpolation approach, starting from nodal coordinates and values.

3.1 General Lagrange Interpolation (arbitrary n)

For n nodes at $x_1, \ldots, x_n$ with values $u_1, \ldots, u_n$, the Lagrange interpolant is:

$$u_h(x) = \sum_{i=1}^{n} \ell_i(x) u_i, \tag{3.1}$$

where the basis (Lagrange) functions are defined as

$$\ell_i(x) = \prod_{\substack{j=1 \\ j \neq i}}^{n} \frac{x - x_j}{x_i - x_j}, \qquad i = 1, \ldots, n. \tag{3.2}$$

Properties (general)

- **Nodal property:** $\ell_i(x_j) = \delta_{ij}$.
- **Partition of unity:** $\sum_{i=1}^{n} \ell_i(x) = 1$.
- **Completeness:** exactly reproduces polynomials up to degree $n - 1$.
- **Coordinate interpolation:** $x = \sum_i x_i \ell_i(x)$ when the ℓ_i are constructed from nodal coordinates.
- **Derivative sum:** $\sum_i d\ell_i/dx = 0$.

1D Linear Shape Functions ($n = 2$, denote by L_i)

Nodes: $x_1 = a$, $x_2 = b$. The two Lagrange functions (linear) are

$$L_1(x) = \frac{x - x_2}{x_1 - x_2} = \frac{b - x}{b - a}, \tag{3.3}$$

$$L_2(x) = \frac{x - x_1}{x_2 - x_1} = \frac{x - a}{b - a}. \tag{3.4}$$

Reference forms (with $\xi \in [-1, 1]$ and $x = \frac{a+b}{2} + \frac{b-a}{2}\xi$):

$$L_1(\xi) = \frac{1 - \xi}{2}, \qquad L_2(\xi) = \frac{1 + \xi}{2}. \tag{3.5}$$

Properties (linear)

- Nodal property: $L_i(x_j) = \delta_{ij}$.
- Partition of unity: $L_1 + L_2 = 1$.
- Linear completeness: reproduces constants and linear functions exactly.
- Coordinate interpolation: $x = x_1 L_1 + x_2 L_2$.
- Derivatives: $\dfrac{dL_1}{dx} = -\dfrac{1}{b-a}$, $\dfrac{dL_2}{dx} = \dfrac{1}{b-a}$ and sum is zero.



Figure 3.1: Reference to physical mapping for 1D linear element.

1D Quadratic Shape Functions ($n = 3$, denote by Q_i)

Nodes: $x_1 = a$, $x_2 = \frac{a+b}{2}$ (midpoint), $x_3 = b$. The quadratic Lagrange functions are

$$Q_1(x) = \frac{(x - x_2)(x - x_3)}{(x_1 - x_2)(x_1 - x_3)}, \tag{3.6}$$

$$Q_2(x) = \frac{(x - x_1)(x - x_3)}{(x_2 - x_1)(x_2 - x_3)}, \tag{3.7}$$

$$Q_3(x) = \frac{(x - x_1)(x - x_2)}{(x_3 - x_1)(x_3 - x_2)}. \tag{3.8}$$

After algebraic simplification (using $x_2 = \frac{a+b}{2}$):

$$Q_1(x) = \frac{(x - \frac{a+b}{2})(x-b)}{0.5(b-a)^2}, \tag{3.9}$$

$$Q_2(x) = -\frac{2(x-a)(x-b)}{(b-a)^2}, \tag{3.10}$$

$$Q_3(x) = \frac{(x-a)(x - \frac{a+b}{2})}{0.5(b-a)^2}. \tag{3.11}$$

In reference coordinate $\xi \in [-1, 1]$ (nodes at $\xi_1 = -1, \xi_2 = 0, \xi_3 = 1$) the compact forms are

$$Q_1(\xi) = \tfrac{1}{2}\xi(\xi - 1), \tag{3.12}$$

$$Q_2(\xi) = 1 - \xi^2, \tag{3.13}$$

$$Q_3(\xi) = \tfrac{1}{2}\xi(\xi + 1). \tag{3.14}$$

Properties (quadratic)

- Nodal property: $Q_i(\xi_j) = \delta_{ij}$ for $\xi_j = -1, 0, 1$.
- Partition of unity: $Q_1 + Q_2 + Q_3 = 1$.
- Quadratic completeness: reproduces polynomials up to degree 2 exactly.
- Coordinate interpolation: $x = \sum_{i=1}^{3} x_i Q_i(x)$.
- Derivative relations: $\sum_{i=1}^{3} dQ_i/d\xi = 0$ in reference; use chain rule for d/dx.

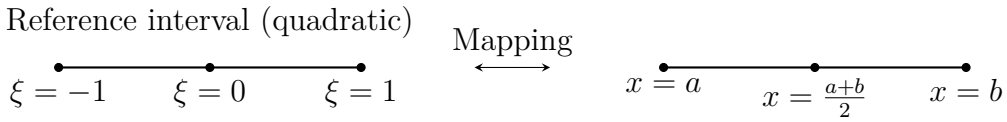


Figure 3.2: Reference to physical mapping for 1D quadratic element (mid-node at $\xi = 0$).

3.2 2D Bilinear Shape Functions ($n = 4$, denote by N_i)

The derivation is performed in the **reference element** (a square) where $\xi, \eta \in [-1, 1]$. The four nodes are numbered counter-clockwise:

- Node 1: $(\xi_1, \eta_1) = (-1, -1)$
- Node 2: $(\xi_2, \eta_2) = (1, -1)$
- Node 3: $(\xi_3, \eta_3) = (1, 1)$
- Node 4: $(\xi_4, \eta_4) = (-1, 1)$

Product of 1D Shape Functions

The 2D bilinear functions are formed by multiplying 1D linear Lagrange functions:

$$N_i(\xi, \eta) = \frac{1}{4}(1 + \xi\xi_i)(1 + \eta\eta_i), \tag{3.15}$$

where (ξ_i, η_i) are nodal coordinates.

Bilinear Shape Functions (N_i)

$$\begin{aligned} N_1(\xi, \eta) &= \frac{1}{4}(1 - \xi)(1 - \eta), \\ N_2(\xi, \eta) &= \frac{1}{4}(1 + \xi)(1 - \eta), \\ N_3(\xi, \eta) &= \frac{1}{4}(1 + \xi)(1 + \eta), \\ N_4(\xi, \eta) &= \frac{1}{4}(1 - \xi)(1 + \eta). \end{aligned} \tag{3.16}$$

Properties (bilinear)

- Nodal property: $N_i(\xi_j, \eta_j) = \delta_{ij}$.
- Partition of unity: $\sum_{i=1}^{4} N_i = 1$.
- Bilinear completeness (in reference): reproduces $\{1, \xi, \eta, \xi\eta\}$ exactly.
- Coordinate interpolation (isoparametric):

$$x(\xi, \eta) = \sum_{i=1}^{4} x_i N_i(\xi, \eta), \qquad y(\xi, \eta) = \sum_{i=1}^{4} y_i N_i(\xi, \eta).$$

- Derivative sums: $\sum_i \partial N_i/\partial\xi = 0$ and $\sum_i \partial N_i/\partial\eta = 0$.

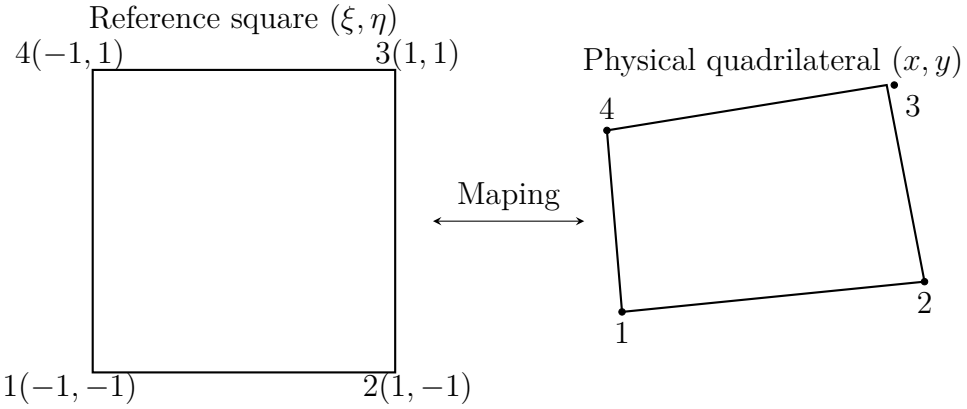


Figure 3.3: Isoparametric mapping from reference square to physical quadrilateral.

Jacobian and integration

The Jacobian matrix is

$$J = \begin{bmatrix} \dfrac{\partial x}{\partial \xi} & \dfrac{\partial y}{\partial \xi} \\ \dfrac{\partial x}{\partial \eta} & \dfrac{\partial y}{\partial \eta} \end{bmatrix}, \quad |J| = \det J.$$

Area elements transform as $dx\,dy = |J|\,d\xi\,d\eta$, and derivatives in physical coordinates are obtained via the inverse Jacobian.

Summary of notation and properties

- General Lagrange basis: $\ell_i(x)$ for arbitrary n.
- Linear 1D basis: L_i (used for $n = 2$).
- Quadratic 1D basis: Q_i (used for $n = 3$).
- Bilinear 2D basis: N_i (used for 4-node quadrilateral).

Each basis satisfies the nodal property, partition of unity, coordinate interpolation, and derivative-sum properties which together ensure compatibility and completeness required for finite element assemblies.

3.3 Evaluation of 1D integral

- Evaluate the following integral using Gaussian Quadrature

$$\int_0^{\frac{\pi}{2}} \sin^2(x)dx$$

1. Exact Analytical Solution

We first determine the exact value of the integral for comparison:

$$I_{\text{exact}} = \int_0^{\frac{\pi}{2}} \sin^2(x)dx$$

Using the trigonometric identity $\sin^2(x) = \frac{1}{2}(1 - \cos(2x))$:

$$I_{\text{exact}} = \frac{1}{2}\left[x - \frac{\sin(2x)}{2}\right]_0^{\frac{\pi}{2}} = \frac{\pi}{4}$$

In decimal form:

$$I_{\text{exact}} \approx 0.7853981634$$

2. Transformation to the Standard Element

To apply the standard Gaussian Quadrature rule, the physical interval $[0, \frac{\pi}{2}]$ must be mapped to the standard element $[-1, 1]$ using linear shape functions.

$$x = x_1 L_1(\xi) + x_2 L_2(\xi)$$

With $x_1 = 0$, $x_2 = \frac{\pi}{2}$, $L_1(\xi) = \frac{1}{2}(1 - \xi)$, and $L_2(\xi) = \frac{1}{2}(1 + \xi)$:

$$x = \frac{\pi}{4}(1 + \xi)$$

The differential relationship is:

$$\frac{dx}{d\xi} = \frac{\pi}{4} \quad \Rightarrow \quad dx = \frac{\pi}{4} d\xi$$

Hence the transformed integral becomes:

$$I = \int_{-1}^{1} \sin^2\left(\frac{\pi}{4}(1 + \xi)\right) \frac{\pi}{4} d\xi$$

where $g(\xi) = \frac{\pi}{4} \sin^2\left(\frac{\pi}{4}(1 + \xi)\right)$.

3. Numerical Evaluation (Gauss–Legendre $n = 3$)

The general Gauss Quadrature rule is $I \approx \sum_{i=1}^{n} u_i g(\xi_i)$. For $n = 3$ Gauss–Legendre:

$$I \approx \frac{5}{9} f\left(-\sqrt{\frac{3}{5}}\right) + \frac{8}{9} g(0) + \frac{5}{9} f\left(\sqrt{\frac{3}{5}}\right) \approx 0.785398$$

4. Comparative Analysis of Quadrature Rules

Table 1: Approximated Results

Table 3.1: Approximated Integral Value ($I \approx \int_0^{\pi/2} \sin^2(x) dx$)

n	**Gauss–Legendre**	**Gauss–Lobatto**	**Gauss–Radau**	**Gauss–Chebyshev**	**Gauss–Laguerre**	**Gauss–Hermite**	**Exact Value**
2	0.785398	0.785398	0.883573	0.872352	0.603130	0.722159	**0.785398**
3	0.785398	0.785398	0.782354	0.822467	0.640200	0.763316	**0.785398**
4	0.785398	0.785398	0.785743	0.805954	0.669734	0.773161	**0.785398**
5	0.785398	0.785398	0.785398	0.795029	0.692348	0.778838	**0.785398**
6	0.785398	0.785398	0.785398	0.790451	0.710198	0.782018	**0.785398**

Table 2: Absolute Errors ($|I_{\text{exact}} - I|$)

Table 3.2: Absolute Error ($|I_{\text{exact}} - I|$)

n	Gauss–Legendre	Gauss–Lobatto	Gauss–Radau	Gauss–Chebyshev	Gauss–Laguerre	Gauss–Hermite
2	1.1×10^{-16}	0.0	9.82×10^{-2}	8.69×10^{-2}	1.82×10^{-1}	6.32×10^{-2}
3	0.0	2.2×10^{-16}	3.04×10^{-3}	3.70×10^{-2}	1.44×10^{-1}	1.71×10^{-2}
4	0.0	0.0	4.47×10^{-5}	2.06×10^{-2}	1.15×10^{-1}	1.22×10^{-2}
5	0.0	0.0	3.20×10^{-7}	9.63×10^{-3}	9.30×10^{-2}	6.56×10^{-3}
6	0.0	0.0	2.10×10^{-9}	5.05×10^{-3}	7.52×10^{-2}	3.38×10^{-3}

Result Analysis

Observing the data in the tables, we can draw the following conclusions:

- Gauss-Legendre gives the closest match to the exact value overall, dropping to machine precision fast.
- Gauss-Lobatto delivers strong precision and stays stable throughout.
- Gauss-Radau works well too, though it converges a touch slower.
- Gauss-Laguerre struggles on this finite interval since it is made for $[0, \infty)$ with specific weights.

3.4 Evaluation of line integral

1. **Problem:** Use a single quadratic element to determine the approximate value of the line integral

$$I = \oint_C dx + 2xdy$$

Here C is the segment of the circle $x^2 + y^2 = 1$ in the first quadrant, defined parametrically by:

$$x = \cos\theta, \quad y = \sin\theta; \quad 0 \leq \theta \leq \frac{\pi}{4}$$

Solution:

The integral to be approximated

The target integral is:

$$I = \int_C dx + 2xdy \quad - \ (1)$$

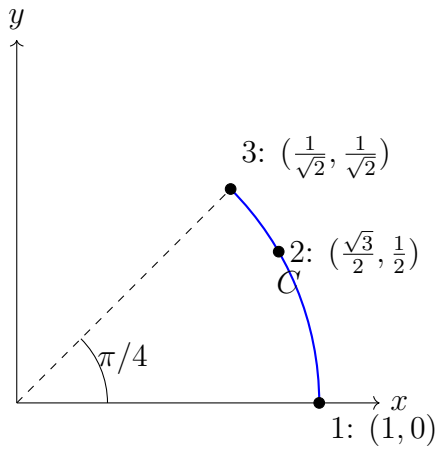


Figure 3.4: The segment C of the unit circle, showing the three nodes and the angle $\pi/4$.

Exact analytical Solution

Using the parametric definitions $x = \cos\theta$ and $y = \sin\theta$, we find the differentials $dx = -\sin\theta d\theta$ and $dy = \cos\theta d\theta$. The exact integral I_{exact} is:

$$\begin{aligned} I_{\text{exact}} &= \int_0^{\pi/4} (-\sin\theta d\theta + 2\cos\theta\cos\theta d\theta) \\ &= \int_0^{\pi/4} (-\sin\theta + 2\cos^2\theta) d\theta \\ &= \left[\cos\theta + \theta + \frac{\sin 2\theta}{2}\right]_0^{\pi/4} \\ &= \frac{-1+\sqrt{2}}{2} + \frac{\pi}{4} \\ &\approx 0.9925049445839959 \end{aligned}$$

Numerical Approximation

To apply Gaussian Quadrature, we approximate the segment C using a single quadratic element defined by 3 nodes:

- Node 1: $(1, 0)$
- Node 2: $(\frac{\sqrt{3}}{2}, \frac{1}{2})$
- Node 3: $(\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}})$ (Corresponds to $\theta = \pi/4$)

Transformation

The line integral is transformed to the standard coordinate $\xi \in [-1, 1]$ using the quadratic shape functions. The quadratic shape functions are:

$$Q_1(\xi) = \frac{1}{2}\xi(\xi - 1); \qquad Q_2(\xi) = 1 - \xi^2; \qquad Q_3(\xi) = \frac{1}{2}\xi(\xi + 1)$$

The coordinate transformation is given by $x = \sum x_i Q_i(\xi)$ and $y = \sum y_i Q_i(\xi)$, where $\xi \in [-1, 1]$.

Using the nodal coordinates, the coordinate transformation equations are:

$$\begin{aligned} x &= 1 \cdot Q_1 + \left(\frac{\sqrt{3}}{2}\right) Q_2 + \left(\frac{1}{\sqrt{2}}\right) Q_3 \\ &\approx -0.12475\xi^2 - 0.14645\xi + 0.8660 \end{aligned}$$

The derivative $\frac{dx}{d\xi}$ is:

$$\frac{dx}{d\xi} \approx -0.2494\xi - 0.14645$$

And for y:

$$\begin{aligned} y &= 0 \cdot Q_1 + \frac{1}{2}Q_2 + \frac{1}{\sqrt{2}}Q_3 \\ &\approx 0.5 + 0.3536\xi - 0.24289\xi^2 \end{aligned}$$

The derivative $\frac{dy}{d\xi}$ is:

$$\frac{dy}{d\xi} \approx 0.3536 - 0.24289\xi$$

Integral in Transformed Coordinate

After transformation, the integral becomes $I = \int_{-1}^{1} \left(\frac{dx}{d\xi} + 2x\frac{dy}{d\xi}\right) d\xi$, resulting in the function $g(\xi)$:

$$I = \int_{-1}^{1} \left(0.0073\xi^3 + 0.7697\xi^2 - 0.6358\xi + 1.4659\right) d\xi$$

$$I = \int_{-1}^{1} g(\xi) d\xi$$

Applying Gaussian Quadrature

Using the Gauss-Legendre rule with $n = 4$:

$$I \approx \sum_{i=1}^{n} u_i g(\xi_i) \approx 0.9831632476$$

The Absolute Error for $n = 4$ Gauss-Legendre is:

$$\text{Absolute Error} = |I_{\text{exact}} - I| = 9.341696 \times 10^{-3}$$

Comparative Results and Analysis

Table of Absolute Errors

The absolute errors for various Gauss-type quadrature methods across different numbers of integration points (n) are compared below:

Table 3.3: Absolute Error Comparison for Different Quadrature Methods ($|I_{\text{exact}} - I|$)

n	**Gauss–Legendre**	**Gauss–Lobatto**	**Gauss–Radau**	**Gauss–Chebyshev**	**Gauss–Laguerre**	**Gauss–Hermite**
5	9.34×10^{-3}	9.34×10^{-3}	9.34×10^{-3}	1.97×10^{-2}	8.92×10^{-3}	1.72×10^{-2}
10	9.34×10^{-3}	9.34×10^{-3}	9.34×10^{-3}	6.33×10^{-3}	4.85×10^{-3}	9.87×10^{-3}
15	9.34×10^{-3}	9.34×10^{-3}	9.34×10^{-3}	6.65×10^{-3}	7.35×10^{-3}	9.42×10^{-3}
20	9.34×10^{-3}	9.34×10^{-3}	9.34×10^{-3}	7.22×10^{-3}	8.22×10^{-3}	9.35×10^{-3}
30	9.34×10^{-3}	9.34×10^{-3}	9.34×10^{-3}	7.90×10^{-3}	8.85×10^{-3}	9.34×10^{-3}

Table 3.4: Absolute Error Comparison for Different Quadrature Methods ($|I_{\text{exact}} - I|$)

n	**Gauss-Legendre**	**Gauss-Lobatto**	**Gauss-Radau**	**Gauss-Laguerre**	**Gauss-Chebyshev**	**Gauss-Hermite**
5	9.34×10^{-3}	9.34×10^{-3}	9.34×10^{-3}	1.97×10^{-2}	8.92×10^{-3}	1.72×10^{-2}
10	9.34×10^{-3}	9.34×10^{-3}	9.34×10^{-3}	6.33×10^{-3}	4.85×10^{-3}	9.87×10^{-3}
15	9.34×10^{-3}	9.34×10^{-3}	9.34×10^{-3}	6.65×10^{-3}	7.35×10^{-3}	9.42×10^{-3}
20	9.34×10^{-3}	9.34×10^{-3}	9.34×10^{-3}	7.22×10^{-3}	8.22×10^{-3}	9.35×10^{-3}
30	9.34×10^{-3}	9.34×10^{-3}	9.34×10^{-3}	7.90×10^{-3}	8.85×10^{-3}	9.34×10^{-3}

Result analysis

The comparative analysis of the quadrature methods' performance yields the following key points:

- Best Initial Performance (small n): Gauss-Legendre, Gauss-Lobatto, and Gauss-Radau lead the pack early on, keeping absolute errors low around 9.34×10^{-3}.
- Overall Best Single Result: Chebyshev hits the lowest error in the table at $n = 10$, with 4.85×10^{-3}.
- Poor Convergence Rate: All methods struggle to improve much after about three decimal places; errors flatten out as n grows.

- Instability at Large n: For $n > 10$, Chebyshev and Laguerre start drifting farther from the exact value, showing instability with higher orders on this integral.

3.5 Evaluation of the Double Integral by Gaussian Quadrature

3.5.1 Rectangular Region

- Evaluate $\int_3^5 \int_1^4 \sqrt{x+y}\, dxdy$

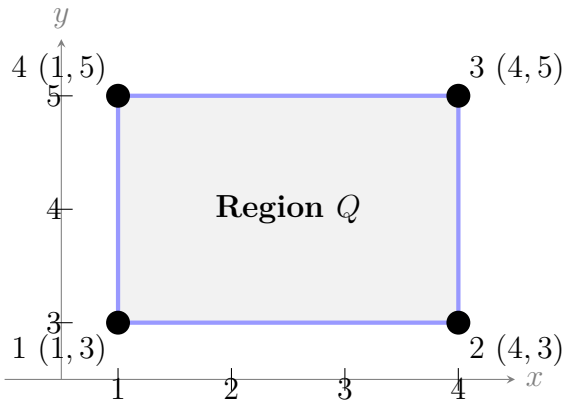


Figure 3.5: Region of Integration Q.

Our aim is to evaluate the following integral over the rectangle region Q:

$$I = \iint_Q \sqrt{x+y}\, dx\, dy$$

The region Q is defined by the following nodes:

1. $(\mathbf{1}, \mathbf{3})$
2. $(\mathbf{4}, \mathbf{3})$
3. $(\mathbf{4}, \mathbf{5})$
4. $(\mathbf{1}, \mathbf{5})$

The exact value of the integral is given as:

$$I_{\text{exact}} = 15.2470813380375$$

Transformation of Region Q to Standard Square

To apply the Gaussian Quadrature rule, we must first transform the physical region Q (where $1 \leq x \leq 4$ and $3 \leq y \leq 5$) to the standard square region in the local coordinate system (ξ, η), where $-1 \leq \xi, \eta \leq 1$. This is achieved using the bilinear shape functions $(N_i(\xi, \eta))$.

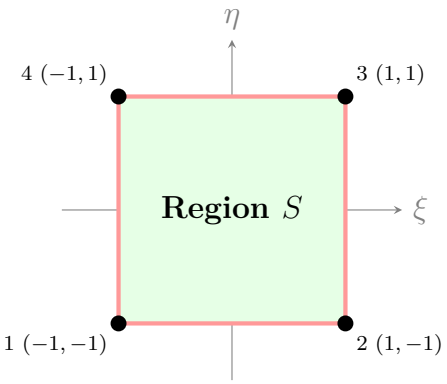


Figure 3.6: The standard square region S in the local (ξ, η) coordinate system, where the integration is performed.

The transformation equations are defined as:

$$x = \sum_{i=1}^{4} x_i N_i(\xi, \eta), \quad y = \sum_{i=1}^{4} y_i N_i(\xi, \eta)$$

where the bilinear shape functions are:

$$N_1(\xi, \eta) = \frac{1}{4}(1 - \xi)(1 - \eta) \qquad N_2(\xi, \eta) = \frac{1}{4}(1 + \xi)(1 - \eta)$$
$$N_3(\xi, \eta) = \frac{1}{4}(1 + \xi)(1 + \eta) \qquad N_4(\xi, \eta) = \frac{1}{4}(1 - \xi)(1 + \eta)$$

Substituting the coordinates of the nodes:

$$x = 1 \cdot N_1 + 4 \cdot N_2 + 4 \cdot N_3 + 1 \cdot N_4$$
$$y = 3 \cdot N_1 + 3 \cdot N_2 + 5 \cdot N_3 + 5 \cdot N_4$$

These expressions simplify to the transformation equations:

$$x = \frac{3}{2}\xi + \frac{5}{2}$$

$$y = \eta + 4$$

Jacobian of the Transformation

The Jacobian determinant $|J|$ is required to change the differential area element $dx\,dy = |J|\,d\xi\,d\eta$.

$$|J| = \det \begin{pmatrix} \frac{\partial x}{\partial \xi} & \frac{\partial x}{\partial \eta} \\ \frac{\partial y}{\partial \xi} & \frac{\partial y}{\partial \eta} \end{pmatrix} = \det \begin{pmatrix} \frac{3}{2} & 0 \\ 0 & 1 \end{pmatrix} = \frac{3}{2}$$

The Transformed Integral

Substituting x and y and the Jacobian into the integral expression:

$$\begin{aligned} I &= \int_{-1}^{1}\int_{-1}^{1} \sqrt{x(\xi,\eta)+y(\xi,\eta)}\,|J|\,d\xi\,d\eta \\ &= \int_{-1}^{1}\int_{-1}^{1} \sqrt{\left(\frac{3}{2}\xi+\frac{5}{2}\right)+(\eta+4)}\cdot\frac{3}{2}\,d\xi\,d\eta \\ &= \int_{-1}^{1}\int_{-1}^{1} g(\xi,\eta)\,d\xi\,d\eta \end{aligned}$$

where the integrand is defined as:

$$g(\xi,\eta)=\frac{3}{2}\sqrt{\frac{3}{2}\xi+\eta+\frac{13}{2}}$$

Application of Gauss-Legendre Quadrature Rule

We apply the two-dimensional Gauss-Legendre Quadrature rule with $n=3$ points in each direction (a total of $3\times 3=9$ points). The rule is:

$$I \approx \sum_{i=1}^{3}\sum_{j=1}^{3} u_i u_j g(\xi_i,\eta_j)$$

For $n=3$ (3-point rule):

- **Nodes (ξ_i,η_j):** $\pm\sqrt{3/5}\approx\pm 0.774597$ and 0.
- **Weights (u_i,u_j):** $5/9\approx 0.555556$ and $8/9\approx 0.888889$.

Gauss-Legendre 3×3 Calculation Table

The approximate value of the integral is:

$$I \approx 15.2470828067$$

Absolute Error for the Gauss-Legendre 3×3 rule is calculated as:

$$\text{Absolute Error} = |I_{\text{exact}} - I| = |15.2470813380375 - 15.2470828067| \approx 1.4686\times 10^{-6}$$

Comparison of Absolute Errors

The absolute errors for different Gauss-type quadrature methods and integration points (n) are summarized below:

Table 3.5: Gauss-Legendre 3×3 Quadrature Calculation

Pt	$\boldsymbol{\xi_i}$	$\boldsymbol{\eta_j}$	$\boldsymbol{g(\xi,\eta)}$	$\boldsymbol{u_i}$	$\boldsymbol{u_j}$	$\boldsymbol{u_i u_j g}$
1	-0.774597	-0.774597	3.204355	0.5556	0.5556	0.988999
2	-0.774597	0	3.466561	0.5556	0.8889	1.711433
3	-0.774597	0.774597	3.706582	0.5556	0.5556	1.144624
4	0	-0.774597	3.529172	0.8889	0.5556	1.772431
5	0	0	3.824265	0.8889	0.8889	3.021641
6	0	0.774597	4.045719	0.8889	0.5556	1.997386
7	0.774597	-0.774597	3.936649	0.5556	0.5556	1.214084
8	0.774597	0	4.152019	0.5556	0.8889	2.050371
9	0.774597	0.774597	4.356846	0.5556	0.5556	1.344706
					Sum (I_{approx})	**15.247083**

Table 3.6: Comparison of Absolute Errors for Various Quadrature Rules ($|I_{\text{exact}} - I|$)

n	**Gauss-Legendre**	**Gauss-Lobatto**	**Gauss-Radau**	**Gauss-Chebyshev**	**Gauss-Laguerre**	**Gauss-Hermite**
3	1.46×10^{-6}	3.04×10^{-4}	2.01×10^{-5}	1.466	3.06	1.82×10^{-1}
5	1.53×10^{-10}	1.75×10^{-8}	1.63×10^{-9}	5.09×10^{-1}	1.81	1.82×10^{-1}
10	8.88×10^{-15}	2.13×10^{-14}	1.21×10^{-12}	1.256×10^{-1}	8.96×10^{-1}	1.67×10^{-2}
15	7.10×10^{-15}	3.55×10^{-14}	8.50×10^{-14}	5.52×10^{-2}	5.93×10^{-1}	2.095×10^{-3}
30	7.10×10^{-15}	5.32×10^{-15}	4.21×10^{-6}	1.39×10^{-2}	2.92×10^{-1}	2.26×10^{-5}

Result Anlysis

From the table, here are the key points on how these quadrature rules converge for this integral:

- Gauss-Legendre converges steadily and gives the smallest absolute error for most point counts, hitting machine precision fast.
- Gauss-Lobatto and Gauss-Radau converge well too, just a bit slower than Gauss-Legendre.
- Gauss-Chebyshev, Gauss-Laguerre, and Gauss-Hermite all converge much more slowly to the true value.
- Gauss-Laguerre performs especially badly, with errors staying large even at $n = 30$.
- Gauss-Radau works fine for $n < 20$ but gives poor results when $n \geq 30$.

3.5.2 Quadrilateral Region

Evaluate:

$$I = \int_1^2 \int_{-1}^{3-x} \sqrt{x^2 + y^2}\, dy\, dx$$

Exact Value: $I_{\text{exact}} \approx 4.16872178857775$

Solution:

The double integral I is defined over a quadrilateral region Q in the (x, y) plane:

$$I = \iint_Q \sqrt{x^2 + y^2}\, dy\, dx$$

1. Define the Region of Integration (Q)

The region Q is bounded by $x = 1$, $x = 2$, $y = -1$, and $y = 3 - x$. The four vertices (nodes) are: 1. $(\mathbf{1}, -\mathbf{1})$ 2. $(\mathbf{2}, -\mathbf{1})$ 3. $(\mathbf{2}, \mathbf{1})$ 4. $(\mathbf{1}, \mathbf{2})$

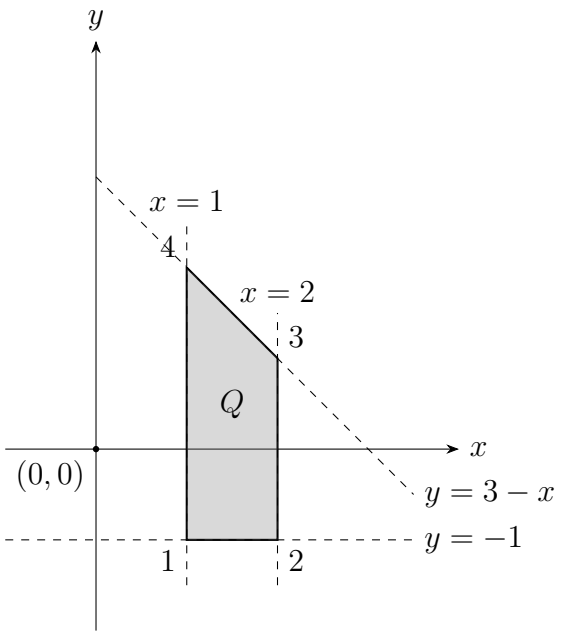


Figure 3.7: Region of Integration Q

2. Transformation

The quadrilateral region Q is mapped to the Standard Square (SS) in the (ξ, η) plane (where $-1 \leq \xi, \eta \leq 1$) using bilinear shape functions, N_i.

$$N_1 = \frac{1}{4}(1 - \xi)(1 - \eta) \qquad N_2 = \frac{1}{4}(1 + \xi)(1 - \eta)$$
$$N_3 = \frac{1}{4}(1 + \xi)(1 + \eta) \qquad N_4 = \frac{1}{4}(1 - \xi)(1 + \eta)$$

Transformation Equations Derivation: The coordinates are mapped using $x(\xi,\eta) = \sum_{i=1}^{4} x_i N_i$ and $y(\xi,\eta) = \sum_{i=1}^{4} y_i N_i$.

$$
\begin{aligned}
x(\xi,\eta) &= \frac{1}{4}[1(1-\xi)(1-\eta) + 2(1+\xi)(1-\eta) + 2(1+\xi)(1+\eta) + 1(1-\xi)(1+\eta)] \\
&= \frac{1}{4}[6 + 2\xi] = \mathbf{\frac{3}{2}} + \mathbf{\frac{\xi}{2}} \\
y(\xi,\eta) &= \frac{1}{4}[-1(1-\xi)(1-\eta) - 1(1+\xi)(1-\eta) + 1(1+\xi)(1+\eta) + 2(1-\xi)(1+\eta)] \\
&= \frac{1}{4}[1 - \xi + 5\eta - \xi\eta] = \mathbf{\frac{1}{4}} - \mathbf{\frac{\xi}{4}} + \mathbf{\frac{5\eta}{4}} - \mathbf{\frac{\xi\eta}{4}}
\end{aligned}
$$

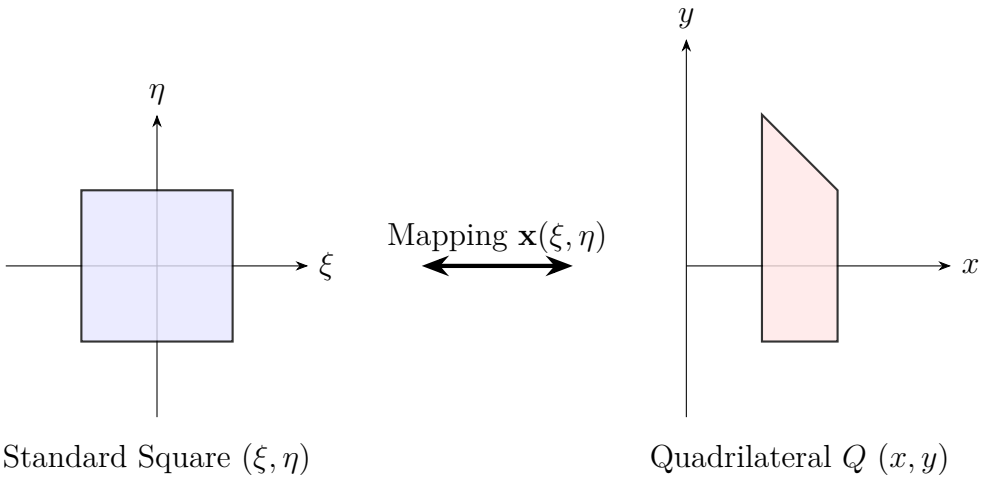


Figure 3.8: Transformation from Standard Square to Quadrilateral Q

3. Calculate the Jacobian Determinant ($|J|$)

The area element transformation is $dx\,dy = |J|\,d\xi\,d\eta$. The Jacobian determinant is:

$$
|J| = \det \begin{vmatrix} \frac{\partial x}{\partial \xi} & \frac{\partial x}{\partial \eta} \\ \frac{\partial y}{\partial \xi} & \frac{\partial y}{\partial \eta} \end{vmatrix}
$$

$$
\begin{aligned}
|J| &= \left(\frac{\partial x}{\partial \xi}\right)\left(\frac{\partial y}{\partial \eta}\right) - \left(\frac{\partial x}{\partial \eta}\right)\left(\frac{\partial y}{\partial \xi}\right) = \left(\frac{1}{2}\right)\left(\frac{5}{4} - \frac{\xi}{4}\right) - (0)\left(-\frac{1}{4} - \frac{\eta}{4}\right) \\
|J| &= \mathbf{\frac{1}{8}}(\mathbf{5} - \xi)
\end{aligned}
$$

Since $-1 \leq \xi \leq 1$, the determinant is always positive.

4. Formulate and Approximate the Transformed Integral

The integral over the standard square becomes:

$$I = \int_{-1}^{1}\int_{-1}^{1} \underbrace{\sqrt{x(\xi,\eta)^2 + y(\xi,\eta)^2} \cdot |J|}_{g(\xi,\eta)}\, d\xi\, d\eta$$

Gauss Quadrature Approximation Applying the product rule for $n \times n$ Gauss Quadrature, the integral I is approximated by a weighted sum of the integrand evaluated at n^2 sampling points:

$$I_n = \sum_{i=1}^{n}\sum_{j=1}^{n} u_i u_j g(\xi_i, \eta_j)$$

Explicit Integrand $g(\xi, \eta)$ Calculation The function $g(\xi, \eta)$ is the product of the square root function evaluated at the transformed coordinates and the Jacobian determinant:

$$\begin{aligned} g(\xi,\eta) &= \sqrt{x(\xi,\eta)^2 + y(\xi,\eta)^2} \cdot |J| \\ &= \sqrt{\left(\frac{3}{2} + \frac{\xi}{2}\right)^2 + \left(\frac{1}{4} - \frac{\xi}{4} + \frac{5\eta}{4} - \frac{\xi\eta}{4}\right)^2} \cdot \left(\frac{1}{8}(5-\xi)\right) \\ &= \frac{1}{8}(5-\xi)\sqrt{\left(\frac{3+\xi}{2}\right)^2 + \left(\frac{(1-\xi)(1+\eta)+4\eta}{4}\right)^2} \end{aligned}$$

Result for 3-Point Gauss-Legendre Quadrature: Using $n = 3$ points for both ξ and η directions:

- **Approximate Value**: $I_{3\times3} \approx 4.1681570495$
- **Absolute Error**: $|I_{\text{exact}} - I_{3\times3}| \approx 5.64 \times 10^{-4}$

Table 3.7: Comparison of Absolute Errors for Different Gauss Quadrature Methods ($|I_{\text{exact}} - I|$)

n	Gauss-Legendre	Gauss-Lobatto	Gauss-Radau	Gauss-Laguerre	Gauss-Hermite	Gauss-Chebyshev
3	5.65×10^{-4}	1.92×10^{-2}	5.16×10^{-3}	5.45×10^{-1}	2.43×10^{-1}	4.46×10^{-1}
5	2.81×10^{-5}	6.43×10^{-5}	9.56×10^{-5}	5.45×10^{-1}	6.73×10^{-2}	1.53×10^{-1}
10	4.63×10^{-9}	6.76×10^{-9}	2.38×10^{-8}	2.85×10^{-1}	4.23×10^{-3}	3.08×10^{-2}
15	5.32×10^{-13}	1.25×10^{-11}	3.69×10^{-11}	1.90×10^{-1}	6.39×10^{-4}	1.66×10^{-2}
30	1.06×10^{-14}	$\mathbf{7.10 \times 10^{-15}}$	1.20×10^{-6}	8.37×10^{-2}	8.24×10^{-6}	4.14×10^{-3}

Result Analysis

- Gauss-Legendre strikes the best balance between accuracy and speed at lower orders, hitting near-machine precision (error of 5.32×10^{-15}) with just $n = 15$.
- Gauss-Lobatto does outstanding work too, ending up with the tiniest error at $n = 30$ points.
- Rules like Gauss-Laguerre, Gauss-Chebyshev, and Gauss-Hermite, built for infinite or semi-infinite intervals or special weights, fall way short here and keep large errors.
- Gauss-Radau shows a big error jump at $n = 30$, hinting at instability for high orders on this integral.

3.5.3 Triangular Region

Problem Statement

The problem is to evaluate the double integral I, given by the iterated integral:

$$I = \int_0^1 \int_0^x e^{|x+y-1|}\, dy\, dx$$

Solution:

This integral is equivalent to the area integral over the region Q:

$$I = \iint_Q e^{|x+y-1|}\, dy\, dx \tag{3.17}$$

The region Q is a triangular region defined by the limits $0 \leq y \leq x$ and $0 \leq x \leq 1$, with vertices: 1. $(0, 0)$ 2. $(1, 0)$ 3. $(1, 1)$
However, for evaluation using Gaussian Quadrature, it is approximated by a quadrilateral defined by the following nodes: 1. (0, 0) 2. (1, 0) 3. (1, 1) 4. $(\frac{1}{2}, \frac{1}{2})$

The exact value is given as:

$$I_{\text{exact}} \approx 0.718281834100361$$

Transformation to Standard Square

To apply the Gauss Quadrature rule, we transform the region Q to the standard square $\Omega = [-1, 1] \times [-1, 1]$ using bilinear shape functions $\mathrm{N}_i(\xi, \eta)$(3.16).

The transformation equations are:

$$x = \sum_i x_i N_i(\xi, \eta) \quad , \quad y = \sum_i y_i N_i(\xi, \eta)$$

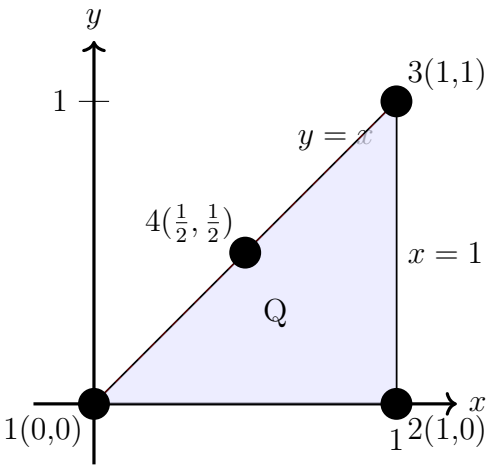


Figure 3.9: Integration region Q, approximated as a quadrilateral for isoperimetric mapping.

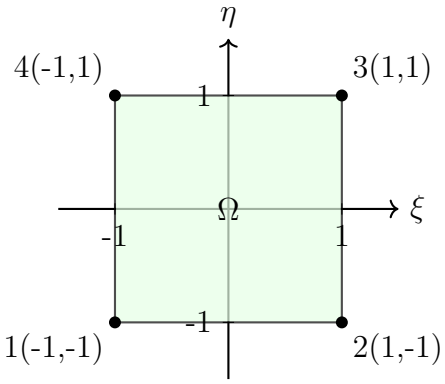


Figure 3.10: Transformation of region Q to the standard square element Ω in $\xi - \eta$ space.

Substituting the nodal coordinates x_i and y_i into the transformation equations yields the parametric expressions for x and y:

$$x(\xi,\eta) = 1 \cdot N_2 + 1 \cdot N_3 + \frac{1}{2} \cdot N_4 = \frac{1}{8}[5 + 3\xi + \eta - \xi\eta]$$
$$y(\xi,\eta) = 1 \cdot N_3 + \frac{1}{2} \cdot N_4 = \frac{1}{8}[3 + \xi + 3\eta + \xi\eta]$$

Jacobian of Transformation

The Jacobian determinant is $|J| = \begin{vmatrix} \frac{\partial x}{\partial \xi} & \frac{\partial x}{\partial \eta} \\ \frac{\partial y}{\partial \xi} & \frac{\partial y}{\partial \eta} \end{vmatrix} = \frac{1}{16}[2 + \xi - \eta]$

The transformation of the integral is:

$$I = \int_{-1}^{1}\int_{-1}^{1} g(x(\xi,\eta), y(\xi,\eta))|J|\, d\xi\, d\eta$$

Transformed Integral

First, we substitute the expressions for $x(\xi, \eta)$ and $y(\xi, \eta)$ to determine the argument of the exponential term in the integrand:

$$x + y = \frac{1}{8}[5 + 3\xi + \eta - \xi\eta] + \frac{1}{8}[3 + \xi + 3\eta + \xi\eta] = \frac{1}{8}[8 + 4\xi + 4\eta] = 1 + \frac{1}{2}(\xi + \eta)$$

Thus, the original function $g(x, y) = e^{|x+y-1|}$ transforms to:

$$g(x(\xi, \eta), y(\xi, \eta)) = e^{\left|(1+\frac{\xi+\eta}{2})-1\right|} = e^{\left|\frac{\xi+\eta}{2}\right|}$$

Hence,

$$g(\xi, \eta)|J| = \left(\frac{2 + \xi - \eta}{16}\right) e^{\left|\frac{\xi+\eta}{2}\right|}$$

Applying Gaussian Quadrature

The Gaussian Quadrature rule is:

$$I \approx \sum_{i=1}^{n} \sum_{j=1}^{n} u_i u_j g(\xi_i, \eta_j)$$

For the Gauss-Legendre rule with $n = 3$:

$$I \approx 0.7070450328$$

The Absolute Error is:

$$\text{Absolute Error} = |I_{\text{exact}} - I| = 1.1236 \times 10^{-2}$$

Table 3.8: Comparison of Absolute Errors for Various Quadrature Rules ($|I_{\text{exact}} - I|$)

n	Gauss-Legendre	Gauss-Lobatto	Gauss-Radau	Gauss-Laguerre	Gauss-Chebyshev	Gauss-Hermite
5	4.53×10^{-3}	7.27×10^{-3}	2.81×10^{-3}	1.01×10^{-1}	2.38×10^{-2}	2.46×10^{-2}
10	1.24×10^{-3}	1.54×10^{-3}	6.89×10^{-4}	4.56×10^{-2}	5.75×10^{-3}	6.15×10^{-3}
15	5.70×10^{-4}	6.56×10^{-4}	3.05×10^{-4}	3.00×10^{-2}	2.59×10^{-3}	3.74×10^{-3}
20	3.26×10^{-4}	3.61×10^{-4}	1.71×10^{-4}	2.26×10^{-2}	1.42×10^{-3}	2.73×10^{-3}
30	1.47×10^{-4}	1.57×10^{-4}	7.59×10^{-5}	1.48×10^{-2}	6.33×10^{-4}	1.81×10^{-3}

Result Analysis

Looking at the absolute errors across different numbers of points (n), no method shows super-fast convergence on its own. That said, Gauss-Legendre, Gauss-Lobatto, and Gauss-Radau

always give much better results than the rest, which improve at a slower pace. For n up to 20, Gauss-Radau has the smallest error and performs the best. In the end, the Legendre, Lobatto, and Radau (for $n \leq 20$) rules are the top picks for getting an accurate value on this integral.

3.5.4 Pentagonal Region

Problem Statement

Evaluate the integral I over the pentagonal region P:

$$I = \iint_P \frac{x^4 + y^3}{1 + x^2}\, dx\, dy$$

The exact value is given as:

$$I_{\text{exact}} = 1.92403054263265$$

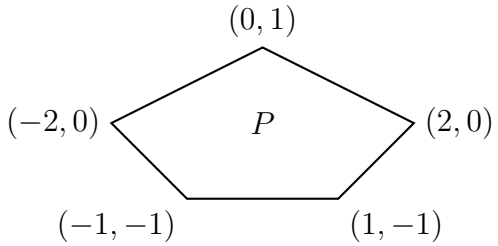


Figure 3.11: Pentagon region P for the integral.

Solution and Discretization

To evaluate the integral over pentagon P, we dissect region P into two quadrilateral elements: element [1] and element [2]. Element [1] consists of nodes $1, 2, 3$ and 4, and element [2] consists of nodes $1, 4, 5$ and 6.

The integral is decomposed as:

$$I = \iint_P \frac{x^4 + y^3}{1 + x^2}\, dx\, dy = \iint_{[1]} \frac{x^4 + y^3}{1 + x^2}\, dx\, dy + \iint_{[2]} \frac{x^4 + y^3}{1 + x^2}\, dx\, dy$$

Let $g(x, y) = \frac{x^4+y^3}{1+x^2}$. Then,

$$I = \iint_{[1]} g(x, y)\, dx\, dy + \iint_{[2]} g(x, y)\, dx\, dy = I_{[1]} + I_{[2]}$$

Element [1] Calculation

Element [1] formed with nodes $1 : (-1, -1), 2 : (1, -1), 3 : (2, 0), 4 : (1, \frac{1}{2})$.

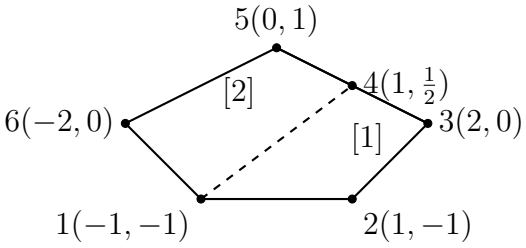


Figure 3.12: Discretization of region P into two quadrilateral elements [1] and [2].

To apply the Gaussian quadrature rule, we convert element [1] to the standard square element $S = [-1, 1] \times [-1, 1]$ using bilinear shape functions, $N_i(\xi, \eta)$(3.16).

The coordinate transformations for element [1] are:

$$x = (-1)N_1 + (1)N_2 + (2)N_3 + (1)N_4$$
$$y = (-1)N_1 + (-1)N_2 + (0)N_3 + \left(\frac{1}{2}\right)N_4$$

The simplified expressions are:

$$x = \frac{3}{4} + \frac{3}{4}\xi + \frac{3}{4}\eta - \frac{1}{4}\xi\eta \quad \text{and} \quad y = -\frac{3}{8} - \frac{1}{8}\xi + \frac{5}{8}\eta - \frac{1}{8}\xi\eta$$

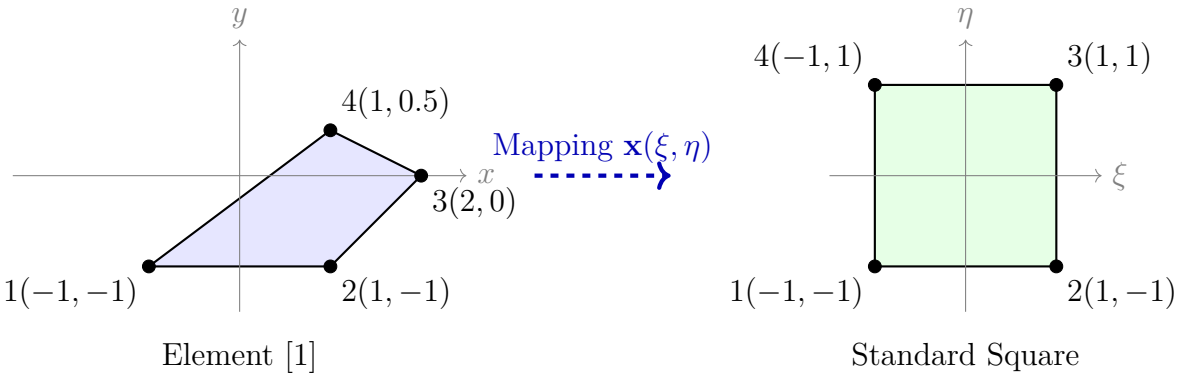


Figure 3.13: Isoparametric transformation (mapping) of Element [1] from the physical (x, y) domain to the standard square (ξ, η) domain.

The Jacobian determinant is defined as:

$$|J| = \det \begin{vmatrix} \frac{\partial x}{\partial \xi} & \frac{\partial x}{\partial \eta} \\ \frac{\partial y}{\partial \xi} & \frac{\partial y}{\partial \eta} \end{vmatrix} = \det \begin{vmatrix} \frac{3}{4} + \frac{\eta}{4} & \frac{3}{4} - \frac{\xi}{4} \\ -\frac{1}{8}(\eta + 1) & \frac{5}{8} - \frac{3\xi}{8} \end{vmatrix}$$

Expanding the determinant gives the simplified Jacobian:

$$|J_{[1]}| = \frac{1}{16}(9 - 2\xi - 1\eta)$$

$$I_{[1]} = \int_{-1}^{1}\int_{-1}^{1} g(x(\xi,\eta), y(\xi,\eta))|J|\, d\xi\, d\eta = \sum_{i=1}^{n}\sum_{j=1}^{n} g(\xi_i, \eta_j)u_i u_j$$

where the integrand evaluated in the standard space is $g(\xi, \eta)$.
The full expression is:

$$g(\xi,\eta) = -\left[2(-5\xi + 3\eta + 3) - (-5\xi - 5\eta + 3)^2\right] \times \frac{(2\xi + \eta - 9) \times 1}{[512(-5\xi + 3\eta + 3)^2 + 8192]}$$

For $n = 3$, applying Gaussian quadrature, the result is:

$$I_{[1]} = 0.7113077423$$

Element [2] Calculation

Element [2] formed with nodes $1 : (-1, -1), 4 : (1, \frac{1}{2}), 5 : (0, 1), 6 : (-2, 0)$.

The coordinate transformations for element [2] are:

$$x = (-1)N_1 + (1)N_2 + (0)N_3 + (-2)N_4$$
$$y = (-1)N_1 + \left(\frac{1}{2}\right)N_2 + (1)N_3 + (0)N_4$$

The simplified expressions (as per the notes) are:

$$x = \frac{3}{8}\xi - \frac{\eta}{2} - \frac{1}{2} \quad \text{and} \quad y = -\frac{5\eta}{8} + \frac{5\xi}{8} + \frac{3\xi\eta}{8} + \frac{1}{8}$$

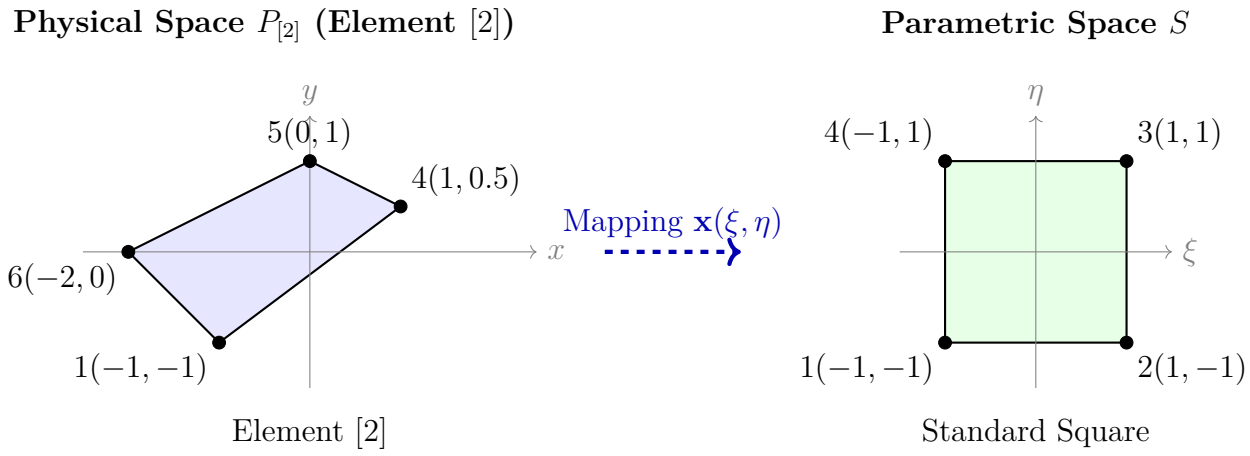


Figure 3.14: Transformation (mapping) of Element [2] from the physical (x, y) domain to the standard square (ξ, η) domain.

Jacobian determinant, The Jacobian matrix:

$$|J| = \det\begin{vmatrix} \frac{\partial x}{\partial \xi} & \frac{\partial x}{\partial \eta} \\ \frac{\partial y}{\partial \xi} & \frac{\partial y}{\partial \eta} \end{vmatrix} = \det\begin{vmatrix} \frac{3}{8} & -\frac{1}{2} \\ \frac{5}{8} + \frac{3\eta}{8} & -\frac{5}{8} + \frac{3\xi}{8} \end{vmatrix}$$

Expanding the determinant gives the simplified Jacobian:

$$|J_{[2]}| = \frac{1}{16}(11 - 2\xi - 1\eta)$$

$$I_{[2]} = \int_{-1}^{1}\int_{-1}^{1} g(x(\xi,\eta), y(\xi,\eta))|J|\, d\xi\, d\eta = \sum_{i=1}^{n}\sum_{j=1}^{n} h(\xi_i, \eta_j)u_i u_j$$

where the integrand evaluated in the standard space is $h(\xi, \eta)$. The full expression is:

$$h(\xi,\eta) = -\left[32(-2\xi + 7\eta)^4 + (-5\xi + 5\eta + 3\eta + 1)^3\right] \times \frac{(2\xi + \eta - 11) \times 1}{2048(-2\xi + 7\eta)^2 + 8192}$$

For $n = 3$, applying Gaussian quadrature, the result is:

$$I_{[2]} = 1.2128264816$$

Final Result and Absolute Error

The approximate integral value I for $n = 3$ is:

$$\therefore I = I_{[1]} + I_{[2]} = 0.7113077423 + 1.2128264816 = 1.9241342240$$

The **Absolute Error** is calculated as:

$$\text{Absolute Error} = |I_{\text{exact}} - I|$$

$$\text{Absolute Error} = |1.92403054263265 - 1.9241342240| = 1.03681349915 \times 10^{-4}$$

Comparison of Quadrature Rules

Similarly, we can form the table for other methods (Absolute Error in $\times 10^{-n}$):

Table 3.9: Comparison of Absolute Errors for Various Quadrature Rules ($|I_{\text{exact}} - I|$)

n	Gauss-Legendre	Gauss-Lobatto	Gauss-Radau	Gauss-Laguerre	Gauss-Chebyshev	Gauss-Hermite
5	1.03×10^{-4}	8.94×10^{-4}	6.91×10^{-5}	5.59×10^{-3}	1.15×10^{-3}	6.96×10^{-2}
10	4.68×10^{-8}	3.36×10^{-9}	6.77×10^{-9}	2.63×10^{-1}	2.73×10^{-2}	2.97×10^{-3}
15	4.03×10^{-13}	2.65×10^{-12}	2.60×10^{-11}	1.71×10^{-1}	6.74×10^{-2}	3.26×10^{-4}
20	2.22×10^{-15}	4.44×10^{-16}	1.07×10^{-9}	1.28×10^{-1}	6.74×10^{-3}	1.42×10^{-4}

Result Analysis

- Low-Order Quadrature ($n \leq 10$): Gauss-Radau and Gauss-Legendre deliver the highest accuracy, and Gauss-Lobatto holds up pretty well too in this range.

- High-Order Quadrature ($n \geq 20$): As n gets larger, Gauss-Lobatto converges the fastest and gives the closest result to the true value at $n = 20$ (absolute error about 4.44×10^{-16}).
- Other Methods: Chebyshev, Hermite, and Laguerre get better with more n, but they converge much slower than the Legendre, Lobatto, and Radau group.
- Gauss-Laguerre Rule: It shows the weakest results and slowest convergence here, probably because its weight function does not fit the transformed integrand.
- Gauss-Radau Rule: Error rises at $n = 20$, pointing to numerical instability for high orders on this integral.

Conclusion

This project presented a comparative study of six Gaussian-based numerical quadrature rules Gauss Legendre, Gauss Lobatto, Gauss Radau, Gauss Laguerre, Gauss Chebyshev, and Gauss Hermite -derived from their respective orthogonal polynomials and weight functions. Among them, the Gauss Legendre quadrature exhibited the highest accuracy and fastest convergence for smooth functions over the finite interval $[-1, 1]$, making it the most efficient general-purpose method. The Gauss Lobatto rule, which includes boundary points, proved stable and suitable for applications such as finite element analysis.

In contrast, the Gauss Radau, Gauss Laguerre, Gauss Chebyshev, and Gauss Hermite quadratures showed reduced or domain-specific accuracy. Overall, Gauss Legendre and Gauss Lobatto emerged as the most reliable and versatile quadrature rules due to their balance of simplicity, stability, and precision. Future work may focus on adaptive and multidimensional Gaussian quadrature extensions to enhance their efficiency in complex computational tasks.

Bibliography

[Stroud1966] Stroud, A. H., Secrest, D. (1966). *Gaussian Quadrature Formulas*. Prentice-Hall.

[Lewis1991] Lewis, P. E., Ward, J. P. (1991). *The Finite Element Method: Principles and Applications*. Addison-Wesley.

[Jain2007] Jain, M. K., Iyengar, S. R. K., Jain, R. K. (2007). *Numerical Methods for Scientific and Engineering Computation*. New Age International.

[Burden2011] Burden, R. L., Faires, J. D. (2011). *Numerical Analysis* (9th ed.). Brooks/Cole, Cengage Learning.

[Hussain2014] Hussain, M. A., Islam, M. S. (2014). Generalized composite numerical integration rule over a polygon using Gaussian quadrature. *Dhaka University Journal of Science*, *62*(1), 25–29.

[Islam2009] Islam, M. S., Hossain, M. A. (2009). Numerical integrations over an arbitrary quadrilateral region. *Applied Mathematics and Computation*, *210*(2), 515–524.

[Islam2008] Islam, M. S., Hossain, M. A. (2008). Numerical integration over an arbitrary triangular region.

[Hildebrand1987] Hildebrand, F. B. (1987). *Introduction to Numerical Analysis* (2nd ed.). Dover Publications.